VAPT Report: Security Assessment of bWAPP

Project Name: Capstone Web Penetration Test

Prepared By: Gracious Akalonu

Period of the project: March 5, 2026 to March 20, 2026

Target: Buggy Web Application (bWAPP) v2.2

TABLE OF CONTENTS

1. Project Title……1
2. Executive Summary……2
3. Scope of Assessment……...3
4. Objectives……4
5. Tools Used……..…5
6. Testing Environment……..6
7. Methodology……7

7.1 Environment Setup……8

7.2 Proxy Configuration (Burp Suite)……..8

7.3 Application Access and Authentication……..…8

7.4 Vulnerability Identification and Exploitation……..8

7.5 Evidence Collection and Documentation……8

8. Vulnerabilities Identified……9

8.1 HTML Injection — Reflected (GET)……..9

8.2 Cross-Site Scripting (XSS)……14

8.2.1 Reflected XSS (GET)……...16

8.2.2 Reflected XSS (Login Form — Error Handling)……17

8.2.3 Stored XSS (User-Agent)……21

8.3 SQL Injection…….23

8.3.1 SQL Injection (GET / Search)……..25

8.3.2 SQL Injection (POST / Search)…….26

8.3.3 SQL Injection (Login Form — Hero)…….28

8.3.4 SQL Injection (Stored — Blog)……32

8.4 OS Command Injection…….34

8.4.1 Command Injection (DNS Lookup)……...35

8.5 Cross-Site Request Forgery (CSRF)……..37

8.5.1 CSRF — Change Password…….38

8.5.2 CSRF — Change Secret……...40

8.6 Broken Authentication…….43

8.6.1 Insecure Login Form (Credentials Exposed in Page Source)……45

9. Risk Severity Summary……46

10. Remediation Recommendations……..48

11. Conclusion……..…….49

2. Executive Summary

This report presents the results of a Vulnerability Assessment and Penetration Testing (VAPT) conducted on the **bWAPP (Buggy Web Application)** platform in a controlled laboratory environment. The purpose of this assessment was to identify security weaknesses, demonstrate how these vulnerabilities can be exploited, and provide recommendations to improve the overall security posture of the application.

The testing process involved setting up a penetration testing environment using **Kali Linux**, deploying the vulnerable application using **Docker**, and utilizing **Burp Suite Professional** to

intercept and analyze HTTP requests between the client and server. The security level of the application was configured to **Low** to allow the identification and exploitation of baseline vulnerabilities commonly found in insecure web applications.

During the assessment, multiple vulnerabilities aligned with the **OWASP Top 10** security risks were successfully identified and exploited. These included **HTML Injection**, **Cross-Site Scripting (XSS)**, **SQL Injection**, **OS Command Injection**, **Cross-Site Request Forgery (CSRF)**, and **Broken Authentication**. Each vulnerability was carefully tested, documented, and supported with evidence such as intercepted requests, payload execution results, and screenshots of successful exploitation.

The findings revealed that the application lacks proper input validation, authentication safeguards, and security controls. These weaknesses could allow attackers to manipulate application behavior, gain unauthorized access, execute system commands, or compromise sensitive data if the system were deployed in a real-world production environment.

Recommendations have been provided throughout this report to address the identified vulnerabilities. Implementing secure coding practices, validating user inputs, enforcing authentication mechanisms, and applying security controls such as CSRF protection and parameterized queries will significantly reduce the risk of exploitation.

Overall, this assessment demonstrates the importance of regular security testing and adherence to secure development standards in protecting web applications from common cyber threats.

3. Scope of Assessment

The scope of this Vulnerability Assessment and Penetration Testing (VAPT) exercise focused on identifying and exploiting security vulnerabilities within the **bWAPP (Buggy Web Application)** in a controlled laboratory environment.

The assessment covered security testing of the web application components to evaluate weaknesses in input validation, authentication mechanisms, session management, and system command handling. Testing activities were performed using industry-standard penetration testing tools and methodologies.

The scope for this security assessment includes, but is not limited to, the following areas:

- Input Validation and Data Handling
- Authentication and Authorization Mechanisms
- Session Management
- Business Logic Testing
- Command Execution and System Interaction
- Web Application Security Controls
- Data Processing and Request Handling

Target Application Details

Application Name:
bWAPP (Buggy Web Application)

Environment:
Local Testing Environment

Accessibility:
Localhost (127.0.0.1:8080)

Deployment Method:
Docker Containers

Testing Type:
Web Application Vulnerability Assessment and Penetration Testing

Out of Scope

The following activities were considered out of scope for this assessment:

- Social Engineering Attacks
- Physical Security Testing
- Network Infrastructure Attacks
- Denial of Service (DoS) Attacks
- Testing of External Systems
- Production Environment Testing

4. Objectives

The primary objective of this Vulnerability Assessment and Penetration Testing (VAPT) exercise was to identify, analyze, and demonstrate security weaknesses within the bWAPP (Buggy Web Application) in a controlled laboratory environment.

The specific objectives of this assessment were to:

- Identify common web application vulnerabilities using practical penetration testing techniques.
- Evaluate the effectiveness of input validation and authentication mechanisms.

- Demonstrate the exploitation of vulnerabilities such as HTML Injection, Cross-Site Scripting (XSS), SQL Injection, OS Command Injection, Cross-Site Request Forgery (CSRF), and Broken Authentication.
- Capture and analyze HTTP requests and responses using Burp Suite.
- Assess the potential impact of identified vulnerabilities on system security.
- Provide remediation recommendations to improve application security.

5. Tools Used

The following tools and technologies were used to perform the Vulnerability Assessment and Penetration Testing (VAPT) on the bWAPP web application. These tools supported the identification, exploitation, and documentation of security vulnerabilities within the controlled testing environment.

1. Kali Linux

2. Burp Suite Professional

3. Docker

4. bWAPP (Buggy Web Application)

5. Web Browser (Burp Suite Embedded Browser)

6. Testing Environment

The Vulnerability Assessment and Penetration Testing (VAPT) exercise was conducted in a controlled laboratory environment to safely identify and exploit security vulnerabilities within the bWAPP web application. The testing environment was configured to simulate a real-world web application setup while ensuring that all activities remained within an isolated and secure system.

The attacker machine and target application were deployed locally using virtualization and containerization technologies to maintain system stability and repeatability during testing.

Environment Configuration

Target Application:

bWAPP (Buggy Web Application)

Deployment Method:

Docker Containers

Operating System (Attacker Machine):

Kali Linux Virtual Machine

Web Application Access:

http://127.0.0.1:8080

Testing Tool:

Burp Suite Professional

Security Level:

Low

Network Type:

Localhost / Isolated Testing Environment

Purpose of the Testing Environment

- The environment was designed to:
- Provide a safe and controlled platform for vulnerability testing
- Simulate real-world web application behavior
- Allow interception and analysis of HTTP traffic
- Enable repeatable testing of security vulnerabilities
- Prevent unintended impact on external systems.

7. Methodology

The Vulnerability Assessment and Penetration Testing (VAPT) was conducted using a structured and systematic approach to identify, exploit, and document security vulnerabilities within the bWAPP web application. The methodology followed standard penetration testing practices to ensure consistent and repeatable testing results within a controlled laboratory environment.

The testing process included environment preparation, proxy configuration, application access, vulnerability testing, and evidence documentation.

7.1 Environment Setup

The testing environment was prepared using virtualization and containerization technologies to simulate a real-world web application testing scenario.

The following setup steps were performed:

- Kali Linux Virtual Machine was launched as the attacker machine.
- Docker was installed and configured on the system.
- The bWAPP application and database were deployed using Docker containers.
- The application was verified to be running successfully.
- The target application was accessed locally using the web browser.

The application was accessible at:

http://127.0.0.1:8080

This confirmed that the testing environment was properly configured and ready for vulnerability assessment.

7.2 Proxy Configuration (Burp Suite)

Burp Suite Professional was configured to intercept HTTP traffic between the web browser and the target application. This allowed requests and responses to be analyzed and modified during testing.

The following steps were performed:

- Burp Suite Professional was launched on the Kali Linux system.
- A new project was created for the penetration testing exercise.
- Proxy interception was enabled using:

Proxy → Intercept → Intercept ON

- Burp Suite's embedded browser was used to access the target application.
- HTTP requests were successfully intercepted before reaching the server.

This confirmed that traffic interception and request monitoring were functioning correctly.

7.3 Application Access and Authentication

After confirming that the proxy was working, the tester accessed the bWAPP web application using the default credentials provided by the system.

The following login credentials were used:

Username: bee

Password: bug

After a successful login:

- The application dashboard containing vulnerability modules was displayed.
- User authentication was confirmed.
- Testing activities were initiated.

7.4 Security Level Configuration

Before beginning vulnerability testing, the application security level was configured to allow baseline vulnerability identification.

The following configuration was applied:

Security Level: Low

Setting the security level to Low ensured that common vulnerabilities could be identified and exploited for learning and demonstration purposes.

7.5 Vulnerability Identification and Exploitation

Multiple web application vulnerabilities were tested using manual penetration testing techniques. Each vulnerability was identified, exploited, and documented using Burp Suite and the web browser.

The testing process involved:

- Entering test payloads into application input fields
- Intercepting HTTP requests using Burp Suite
- Observing application responses
- Confirming successful exploitation of vulnerabilities
- Capturing screenshots as evidence

The following vulnerability categories were successfully tested:

- HTML Injection
- Cross-Site Scripting (XSS)
- SQL Injection
- OS Command Injection
- Cross-Site Request Forgery (CSRF)
- Broken Authentication

7.6 Evidence Collection and Documentation

All successful vulnerability exploitations were documented to support the findings of the assessment.

Evidence collected during testing included:

- Screenshots of vulnerability testing pages
- Captured HTTP requests using Burp Suite
- Payload execution results
- System command outputs
- Error messages returned by the application

This documentation ensured that each vulnerability finding was supported by verifiable technical evidence.

8. Vulnerabilities Identified

During the Vulnerability Assessment and Penetration Testing (VAPT) of the bWAPP web application, multiple security vulnerabilities were identified and successfully exploited within the controlled testing environment. These vulnerabilities demonstrate weaknesses in input validation, authentication mechanisms, and security controls implemented within the application.

Each vulnerability was tested using manual penetration testing techniques and verified through observed system responses and captured HTTP requests using Burp Suite. The findings presented in this section include detailed descriptions of the vulnerabilities, steps to reproduce the attacks, evidence of successful exploitation, potential security impacts, and recommended remediation measures.

The vulnerabilities identified during this assessment are categorized based on common web application security risks aligned with the OWASP Top 10 framework.

8.1 HTML Injection — Reflected (GET)

HTML Injection is a type of web vulnerability where an attacker can inject arbitrary HTML code into a web page. This can modify how the website looks or behaves for the user. It's different from Cross-Site Scripting (XSS) because HTML Injection doesn't execute JavaScript (at least in its basic form). However, it can still be dangerous. Attackers can deface the site, trick users with fake forms, or manipulate the UI.

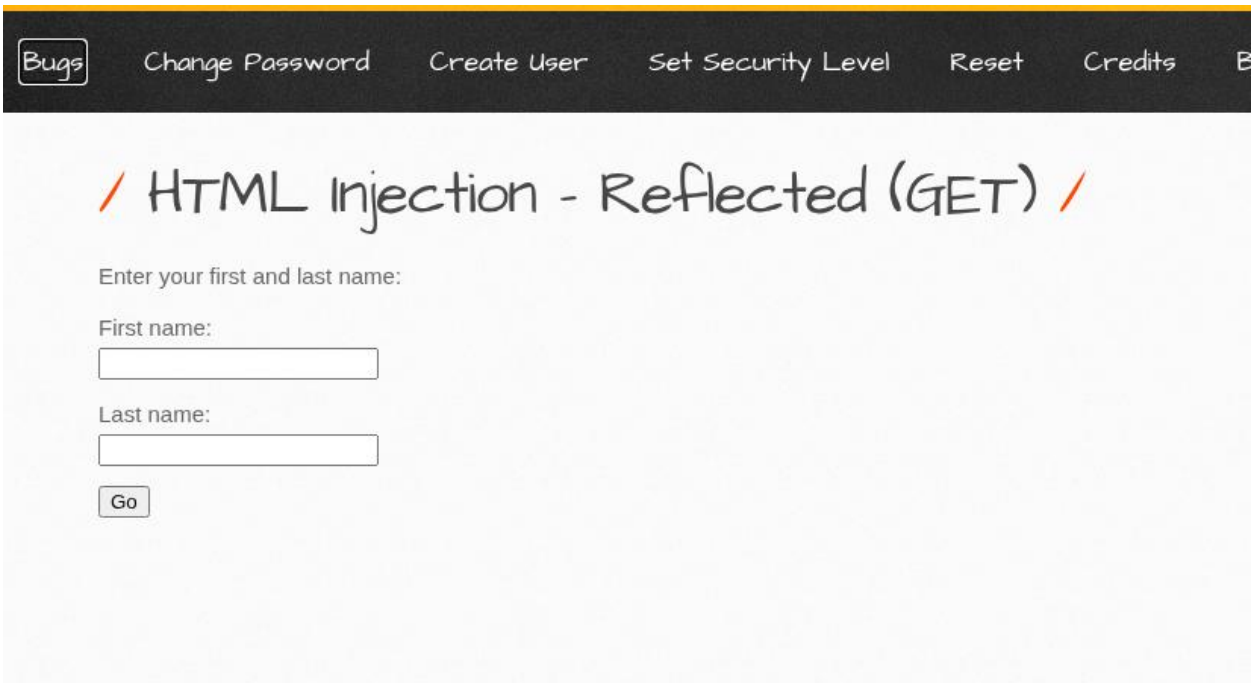


Figure: HTML Injection – Reflected (GET) vulnerability testing page.

Description

The application reflects user input directly into the webpage without proper sanitization. This allows attackers to inject HTML code that will be rendered by the browser.

Steps to Reproduce

1. Navigate to **HTML Injection – Reflected (GET)** in bWAPP.
2. Enter the following payload in the **First Name** field:

```
<h1>Gracious</h1>
```

3. Enter any value in the **Last Name** field.
4. Click **Go**.

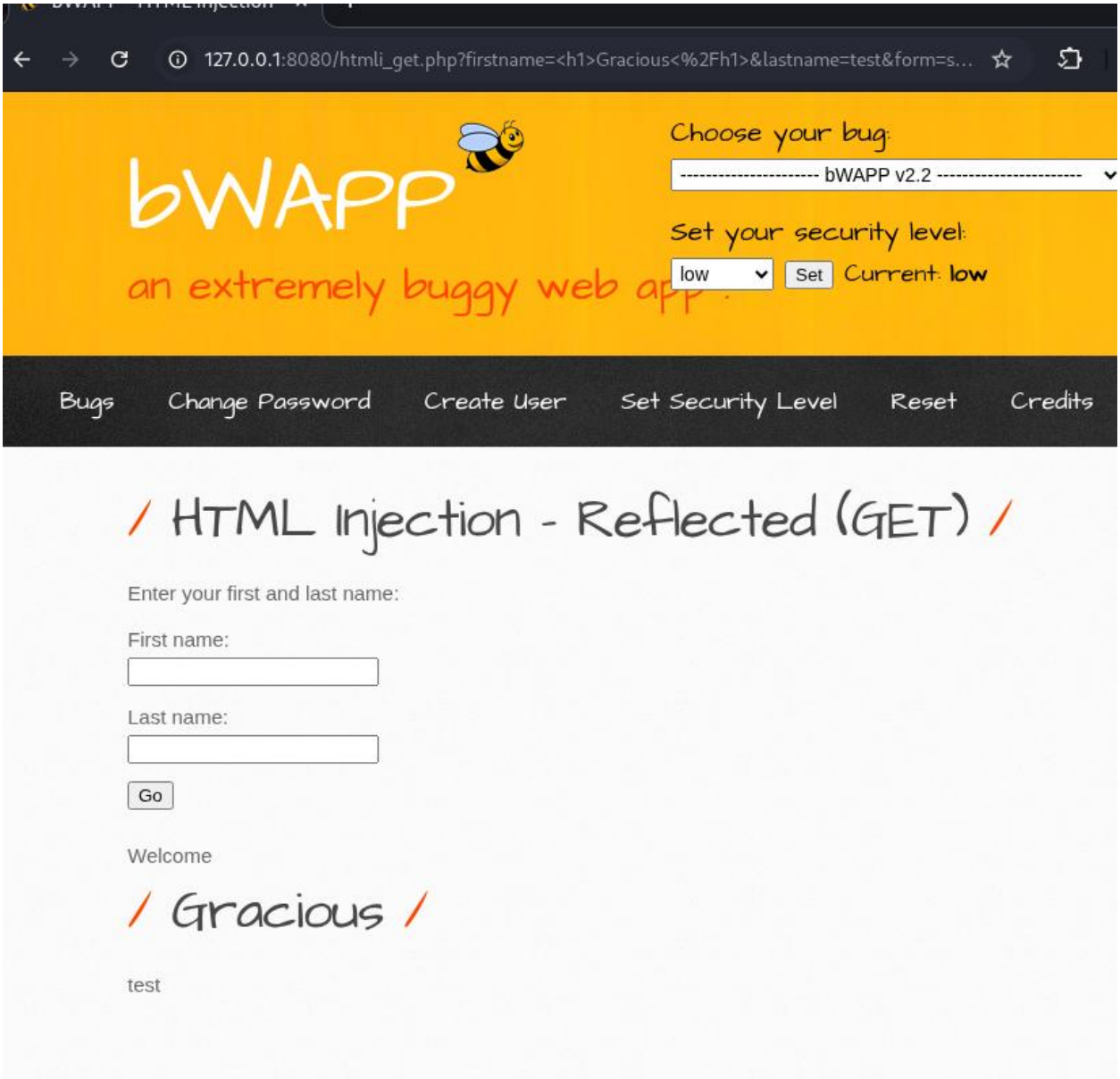
Result

The injected HTML tag `<h1>` was executed by the browser and displayed as a large heading on the page.

Impact

Attackers can inject malicious HTML into the application, which can alter the appearance of the page or trick users into interacting with malicious content.

Evidence



Remediation

User inputs should be sanitized and encoded before being displayed in the browser to prevent HTML injection attacks.

TESTING STEPS

In pentesting, we start with **simple payloads** like:

<h1> </h1>,

before trying advanced attacks. This is called **initial payload testing**.

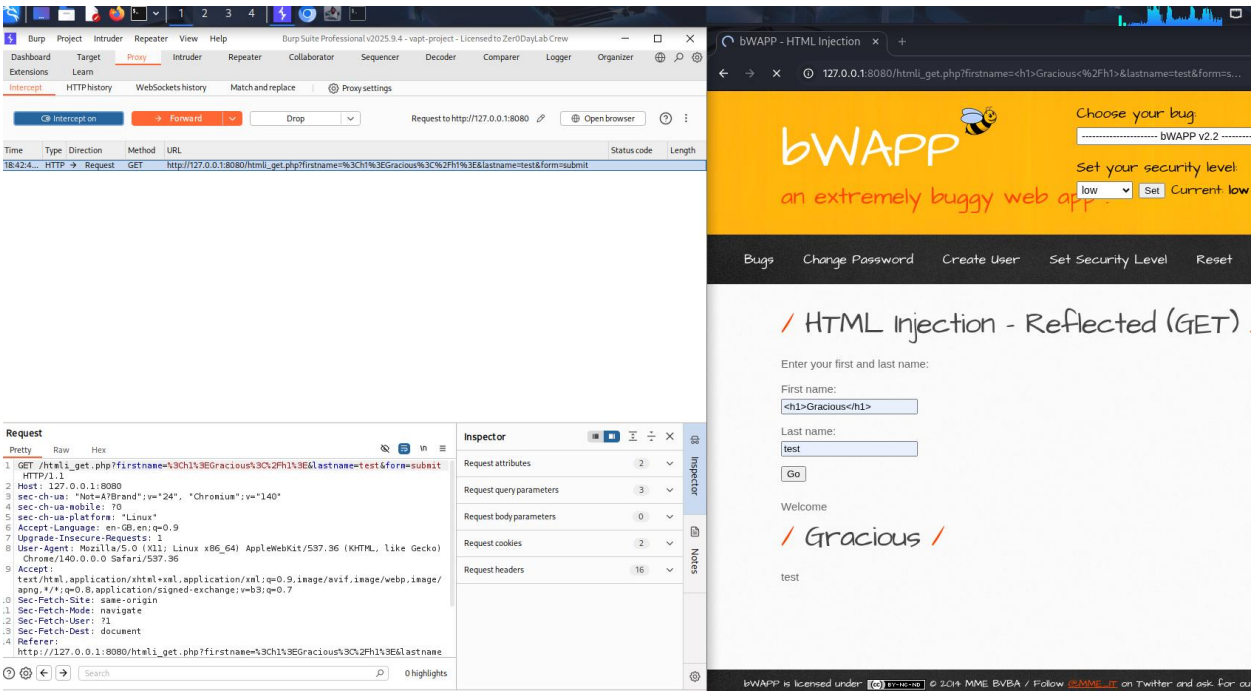
A simple HTML payload <h1>Gracious</h1> was injected into the input field to determine whether the application renders user input without sanitization.

Initial test with only one parameter failed because the application required both input fields. The test was repeated with both fields filled to allow the request to be processed.

The payload was visible in the URL:

htmli_get.php?firstname=<h1>Gracious</h1>&lastname=test

The application reflected the input directly into the webpage without filtering or encoding the HTML tags. As a result, the browser interpreted the <h1> tag and rendered it as a heading.

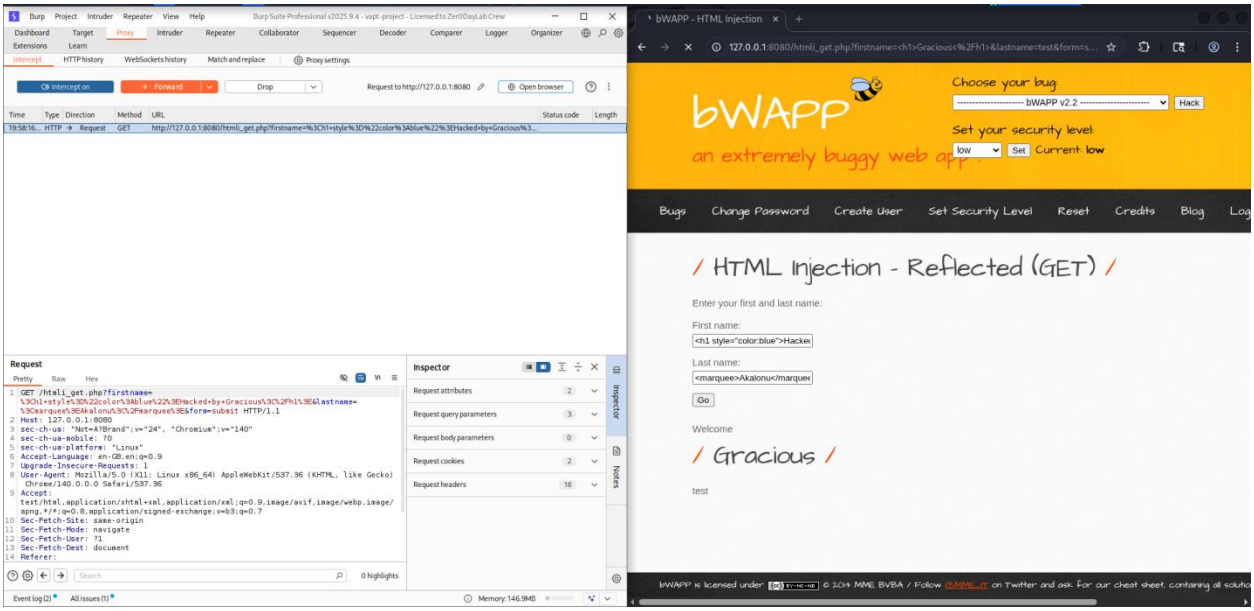


The malicious payload was intercepted using **Burp Suite Proxy**.

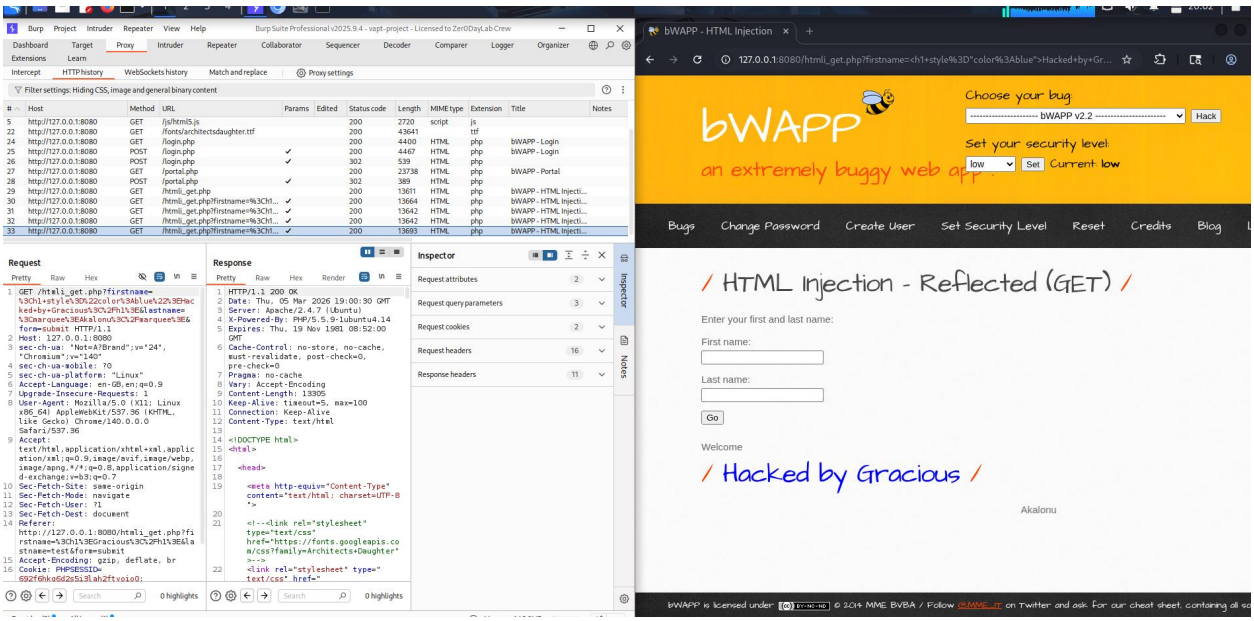
captured request:

GET /htmli_get.php?firstname=<h1>Gracious</h1>&lastname=test&form=submit HTTP/1.1
Host: 127.0.0.1:8080

This confirms the application accepts unsanitized input via the **GET parameter firstname**.

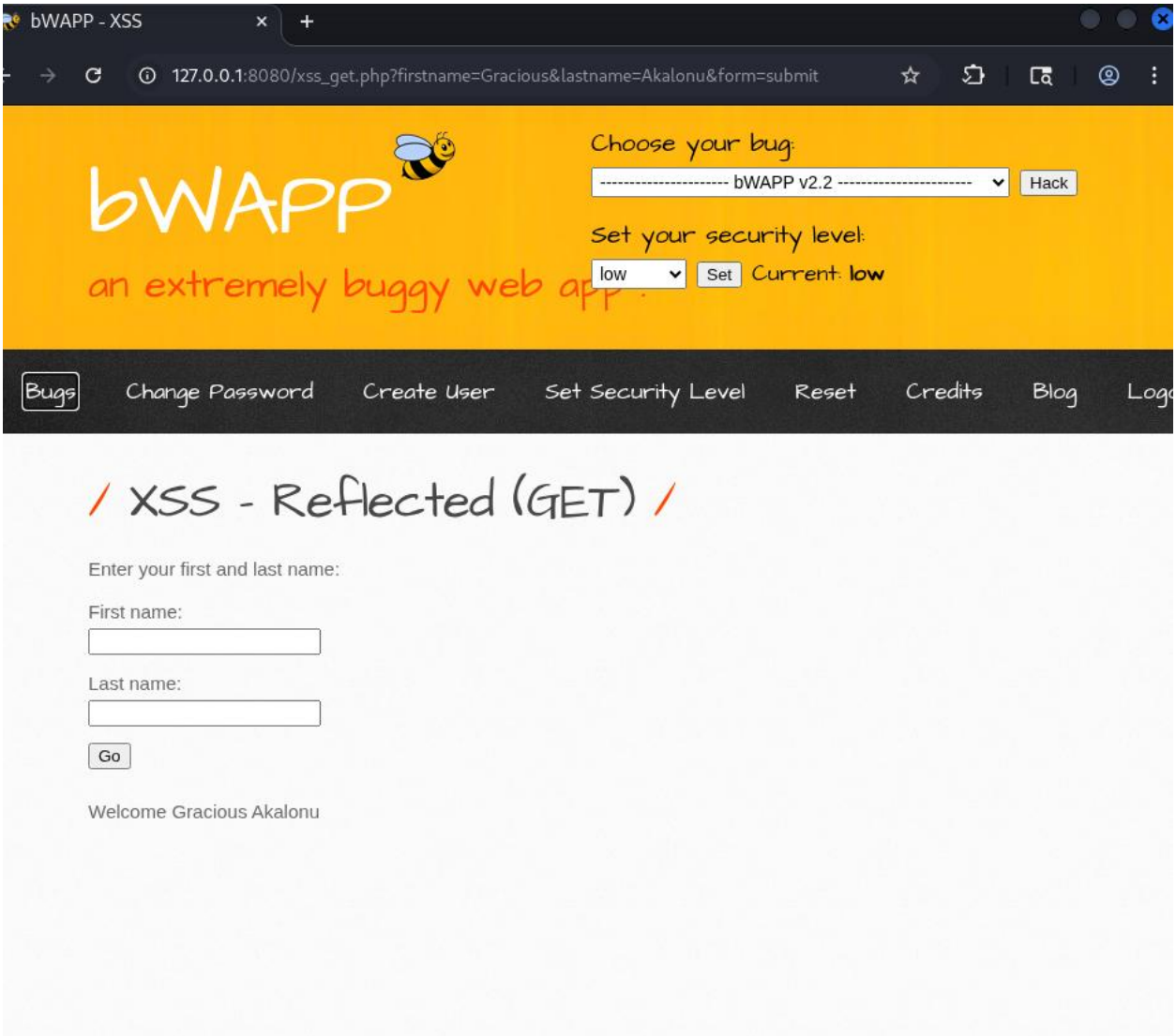


Successful Html Injection exploitations



8.2 Cross-Site Scripting (XSS)

- Reflected XSS (GET)

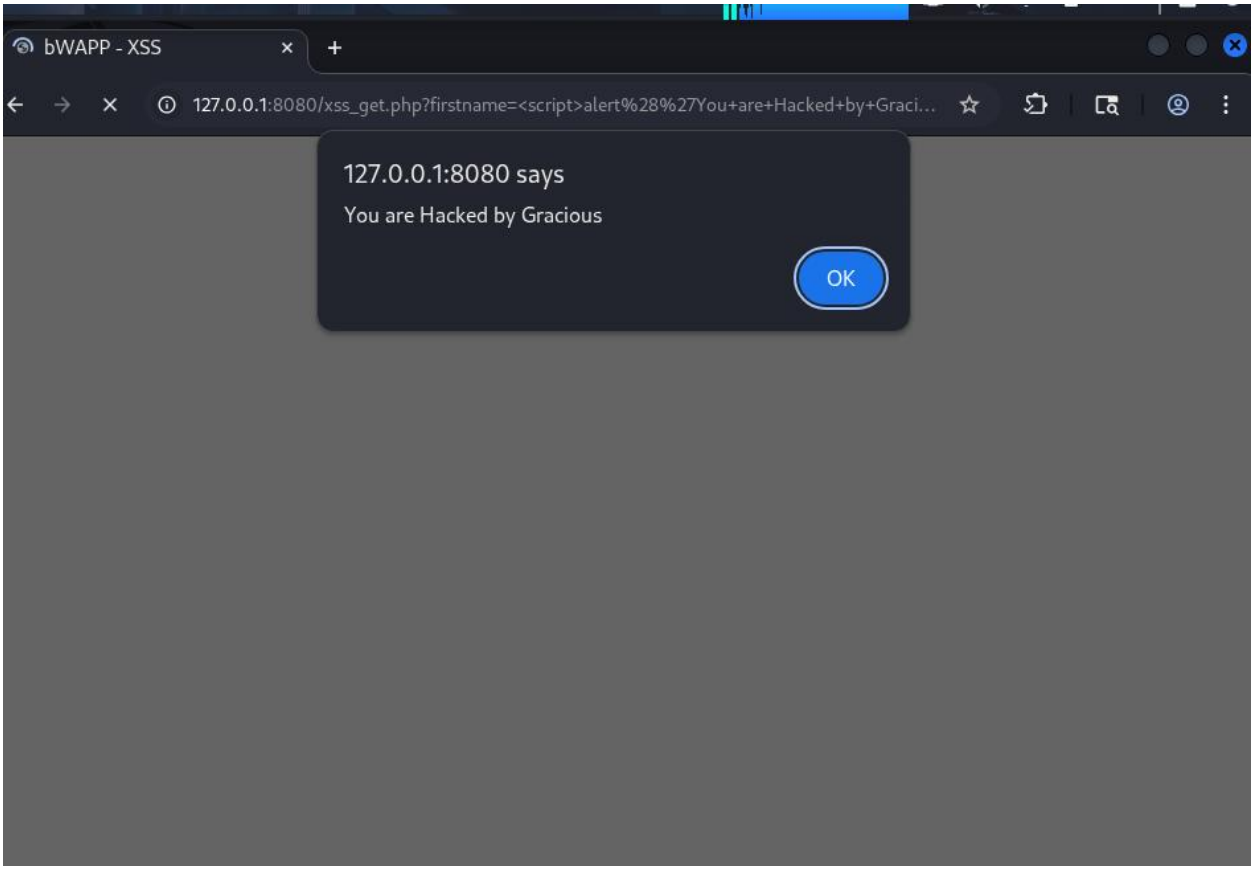


Normal Functionality

The application allows users to input their first and last name through a form.
Upon submission, the application reflects the input back to the user by displaying a welcome message.

Observation

The application directly displays user-supplied input without any visible filtering or sanitization.



Severity: Medium

Description

The application reflects user input directly into the webpage without proper sanitization. This allows an attacker to inject and execute malicious JavaScript code in the victim's browser.

Steps to Reproduce

1. Navigate to **XSS – Reflected (GET)** in bWAPP
2. Enter the following payload in the first name field:

```
<script>alert('You are Hacked by Gracious')</script>
```

3. Enter any value in the last name field
4. Click **Go**

Result

A JavaScript alert box appeared with the message:

"You are Hacked by Gracious"

This confirms that the injected script was executed in the browser.

Impact

An attacker can:

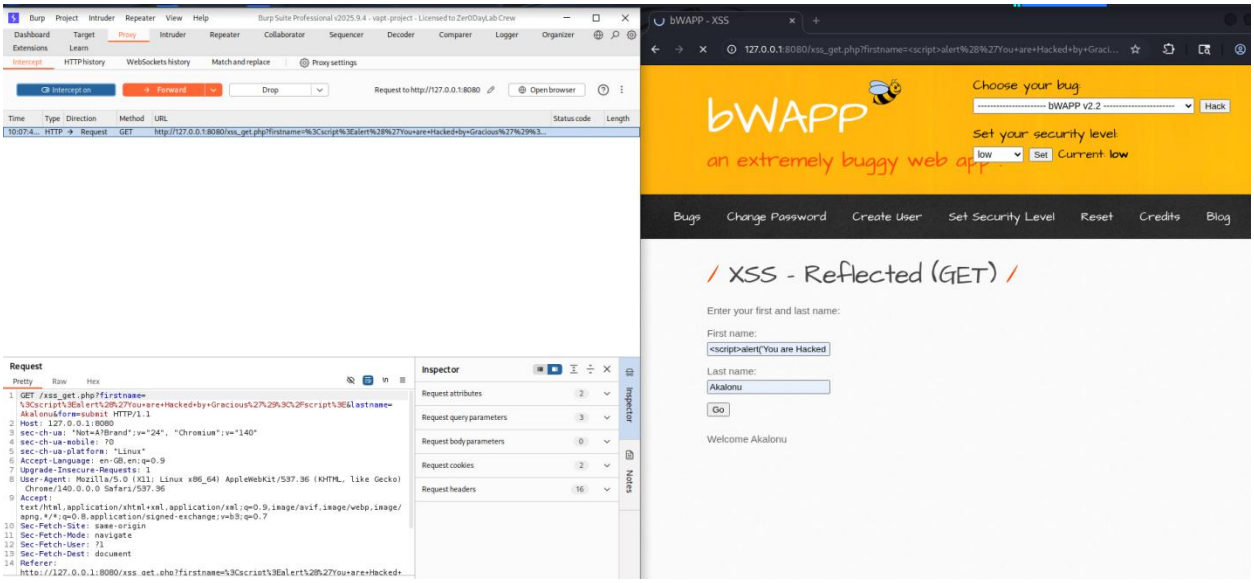
- steal session cookies
- hijack user sessions
- redirect users to malicious websites
- perform actions on behalf of the user

Remediation

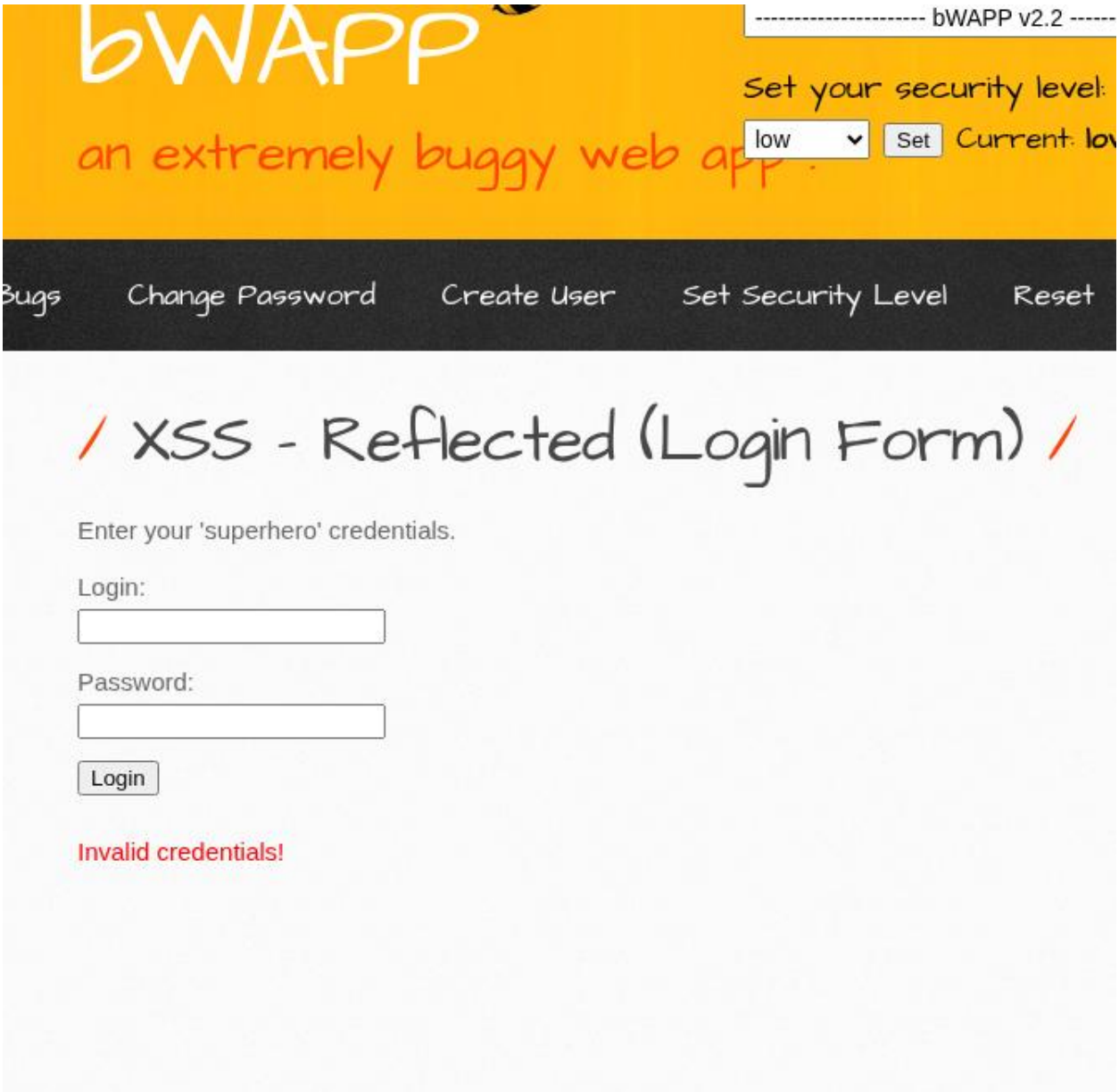
- Sanitize user input
- Encode output before rendering
- Implement Content Security Policy (CSP)

Evidence (Burp Request)

The malicious payload was captured in Burp Suite within the HTTP GET request.
The application accepts and processes the injected script without any input validation.



- Reflected XSS (Login Form — Error Handling)



Normal Functionality

The application provides a login form where users enter a username and password.
When incorrect credentials are submitted, the application responds with an error message:
"Invalid credentials!"

bWAPP

an extremely buggy web app !

Choose your bug:

---------- bWAPP v2.2 ----------

Set your security level:

low Set Current: **low**

Bugs | Change Password | Create User | Set Security Level | Reset | Credits

/ XSS - Reflected (Login Form) /

Enter your 'superhero' credentials.

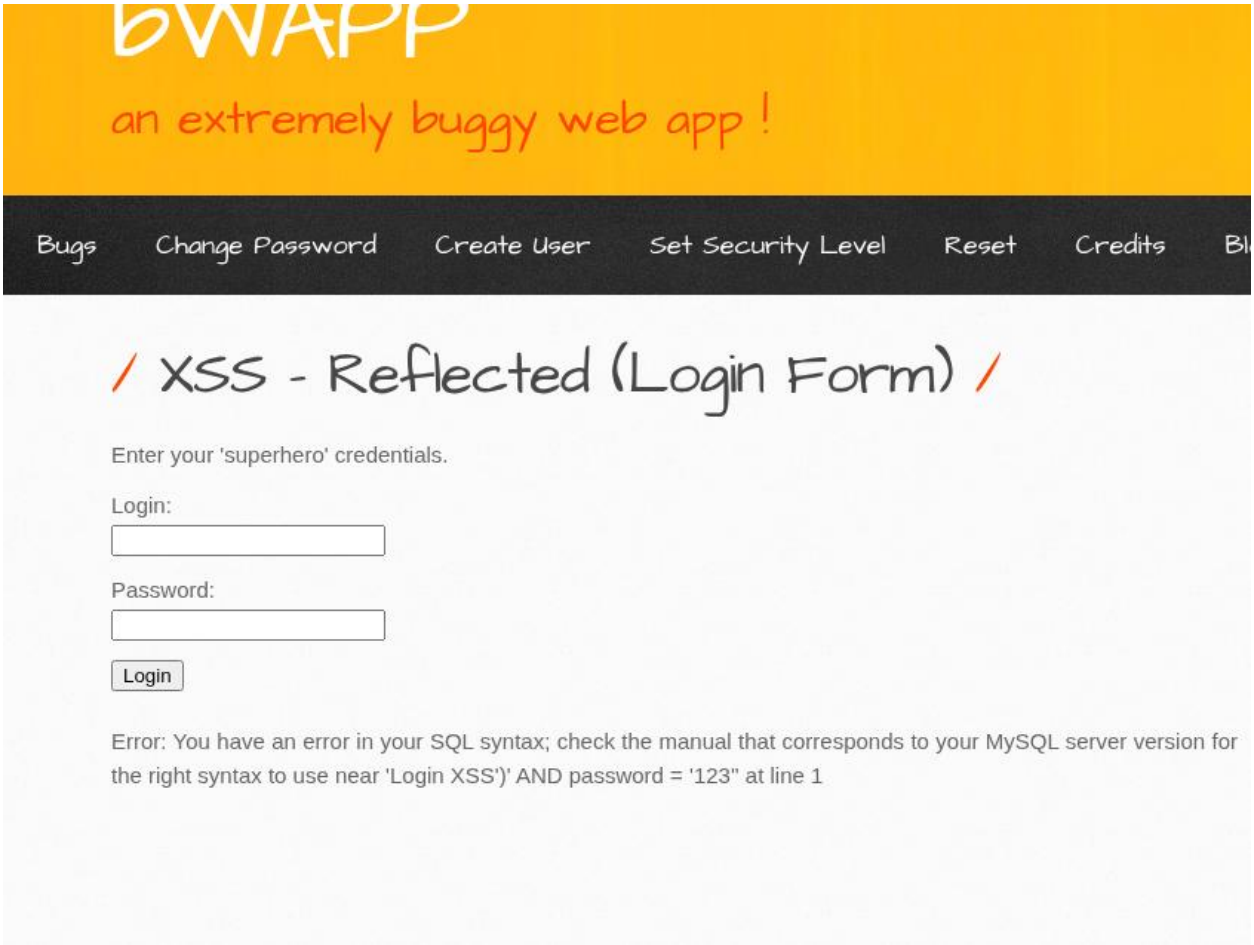
Login:

<script>alert('Login XSS')</sci

Password:

•••

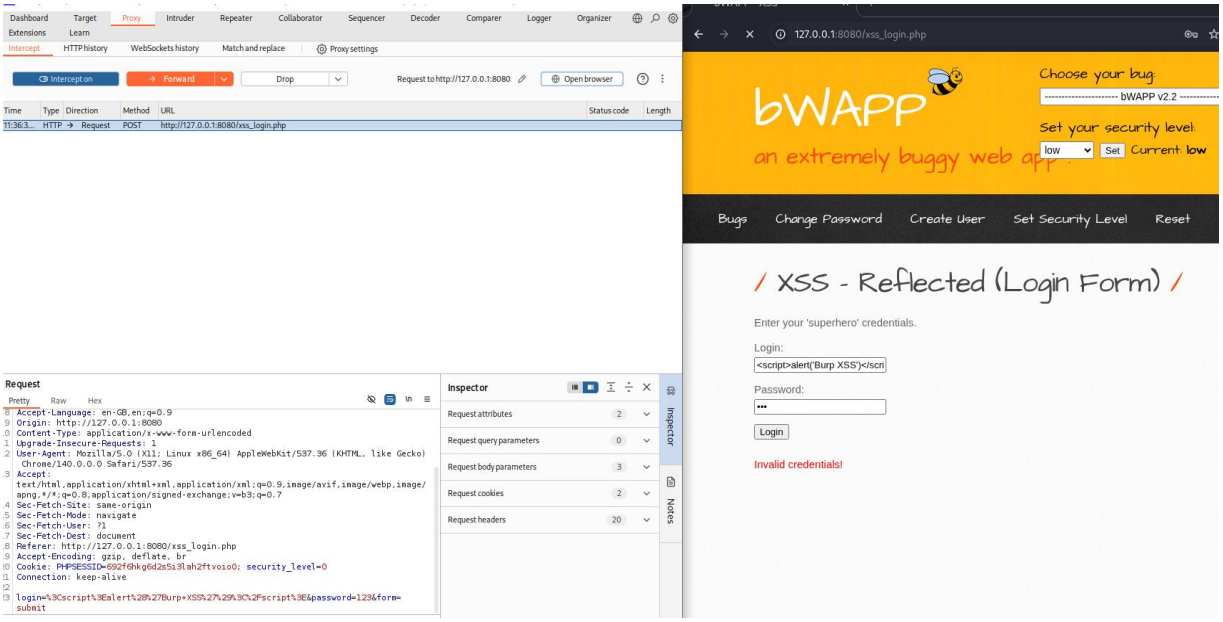
Login



The intended XSS did not execute, but the input caused a backend SQL error, revealing another vulnerability.

Expected	What you found
XSS	❌ Not working
SQL error leak	✅ Vulnerability
SQL Injection potential	Possible

While testing for reflected XSS on the login form, the injected payload did not execute. However, the application returned a detailed SQL error message, indicating improper input handling and potential SQL Injection vulnerability.



Description:

The login form accepts user input and directly processes it in backend operations without proper sanitization. This leads to SQL errors when malformed input is supplied.

Evidence:

The payload was captured in Burp Suite within a POST request:

login=<script>alert('Burp XSS')</script>&password=123

Result:

The application failed to execute the script and instead returned an error, indicating improper handling of input and potential SQL interaction.

Impact:

- Reveals backend behavior
- May assist in SQL Injection attacks
- Indicates lack of input validation

- Stored XSS (User-Agent)

Description:

The application stores user-controlled input from the User-Agent header without proper sanitization. This allows attackers to inject malicious scripts that execute whenever the data is viewed.

Payload:

<script>alert('Stored XSS')</script>

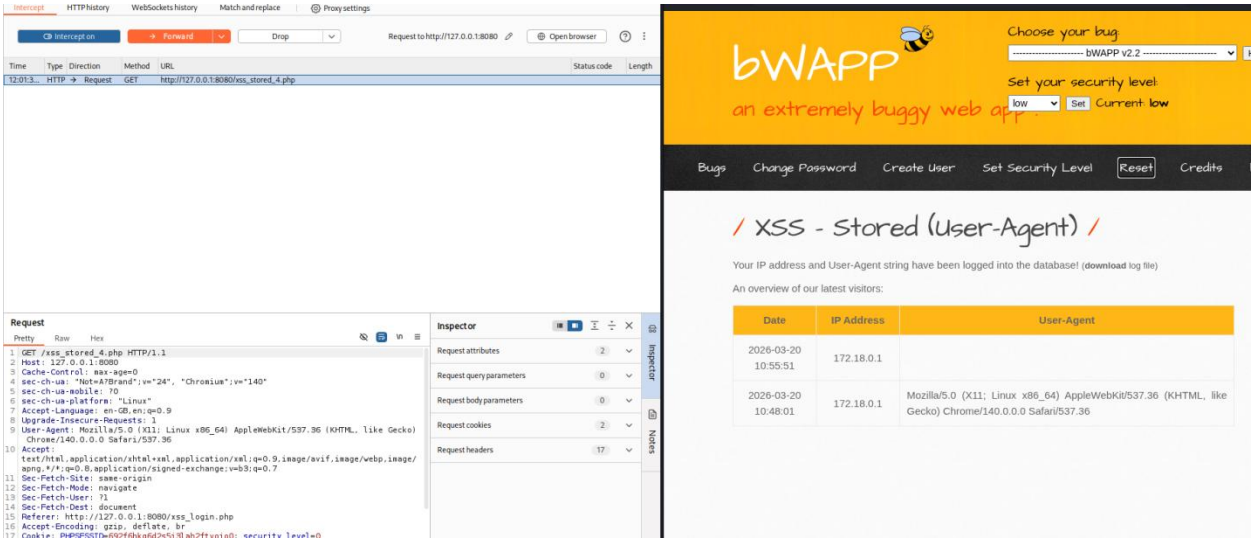
Result:

The payload was stored and executed when the page displaying user agents was loaded, triggering a JavaScript alert.

Impact:

- Affects multiple users
- Can execute automatically
- Can steal session cookies
- Can compromise admin accounts

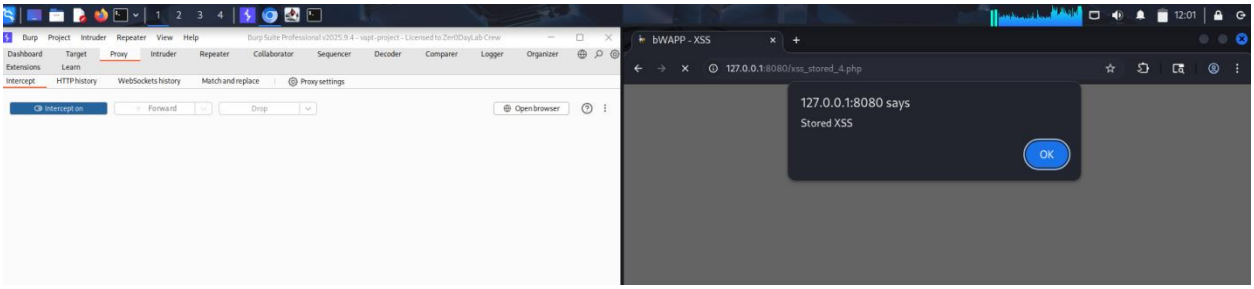
Severity: High

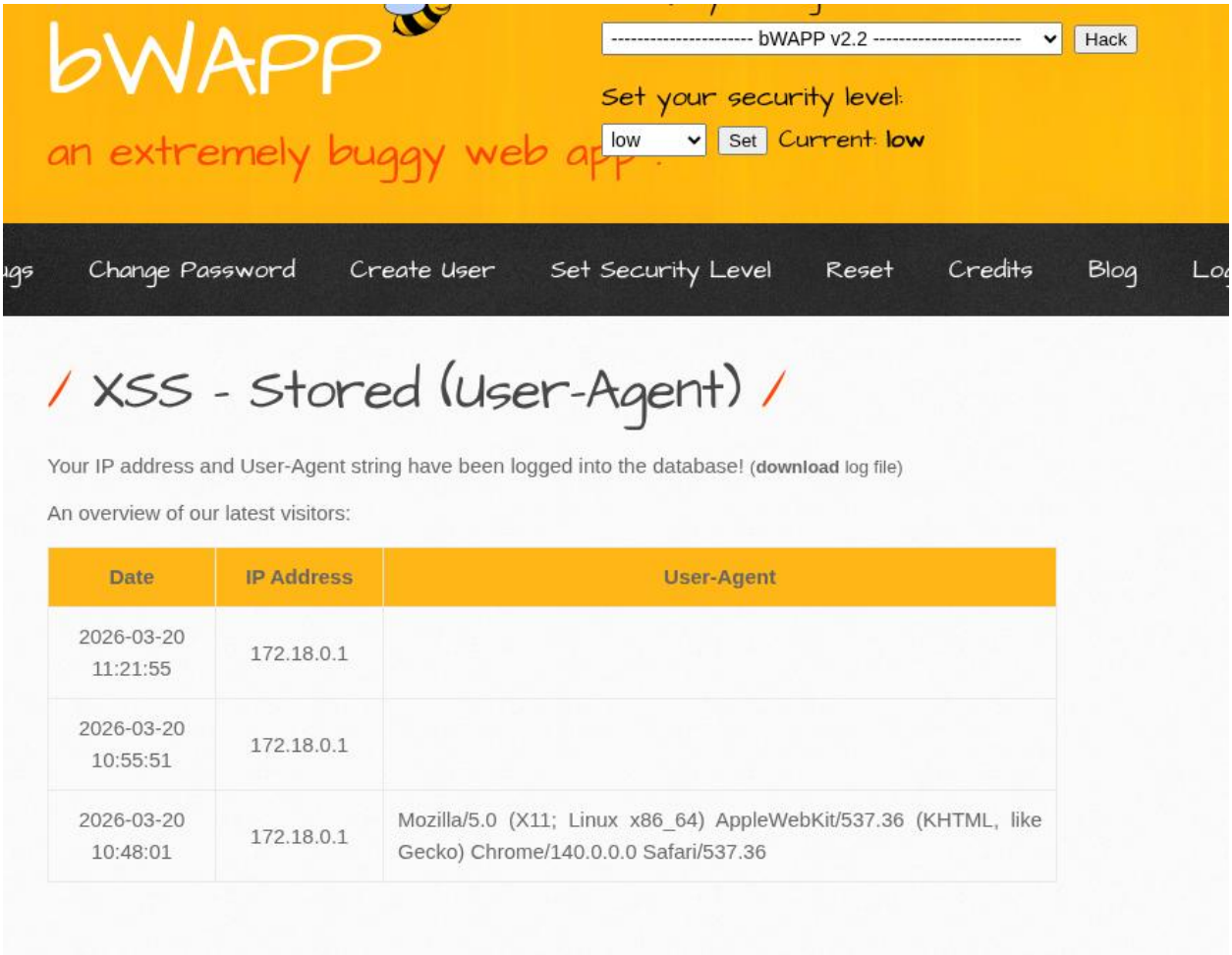


Request

Pretty Raw Hex

```
1  GET /xss_stored_4.php HTTP/1.1
2  Host: 127.0.0.1:8080
3  Cache-Control: max-age=0
4  sec-ch-ua: "Not=A?Brand";v="24", "Chromium";v="140"
5  sec-ch-ua-mobile: ?0
6  sec-ch-ua-platform: "Linux"
7  Accept-Language: en-GB,en;q=0.9
8  Upgrade-Insecure-Requests: 1
9  User-Agent: <script>alert('Stored XSS')</script>
10 Accept:
   text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image,
   apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer: http://127.0.0.1:8080/xss_login.php
16 Accept-Encoding: gzip, deflate, br
17 Cookie: PHPSESSID=692f6hkg6d2s5i3lah2ftvoio0; security_level=0
```





8.3 SQL Injection

- **SQL Injection (GET/Search)**

Severity: Critical

Normal Functionality

The application provides a search feature where users can search for movies by title.
When a valid movie name such as "Iron Man" is entered, only the matching result is returned.

Vulnerability Description

The application fails to properly validate user input in the search parameter.
User input is directly included in an SQL query, allowing attackers to manipulate the query logic.

Payload Used: ' OR 1=1#

Steps to Reproduce

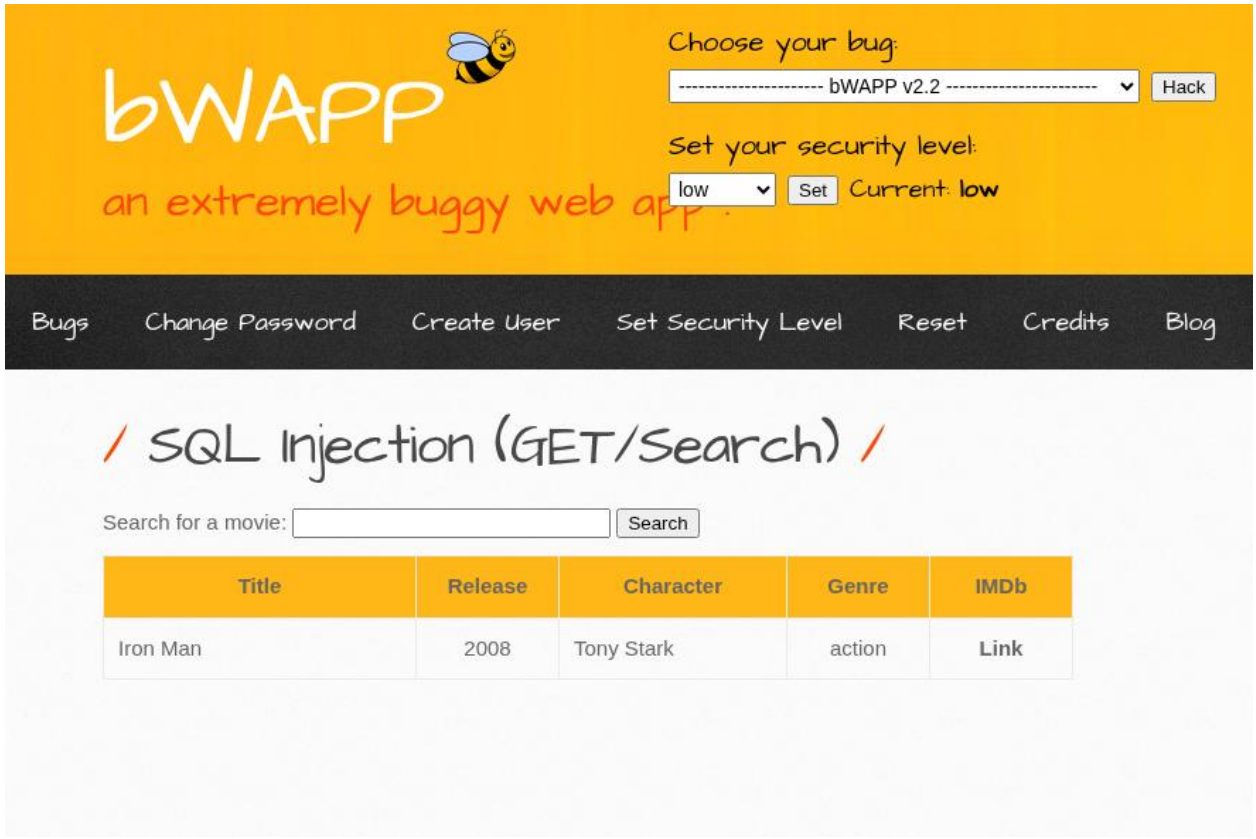
1. Navigate to SQL Injection (GET/Search)
2. Enter the payload ' OR 1=1# in the search field
3. Click **Search**

Result

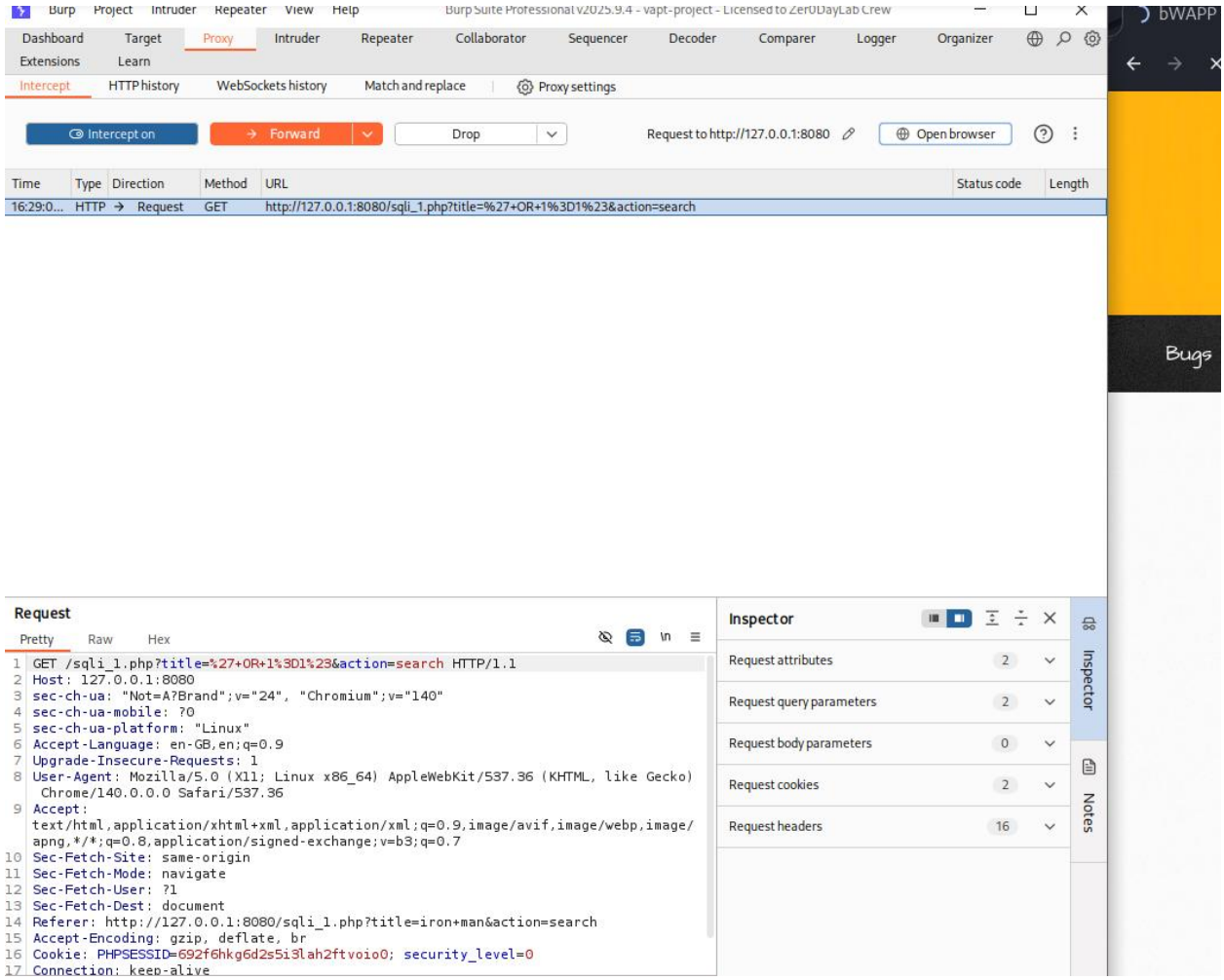
The application returned all movies in the database instead of filtering results.
This confirms that the SQL query logic was bypassed.

Evidence

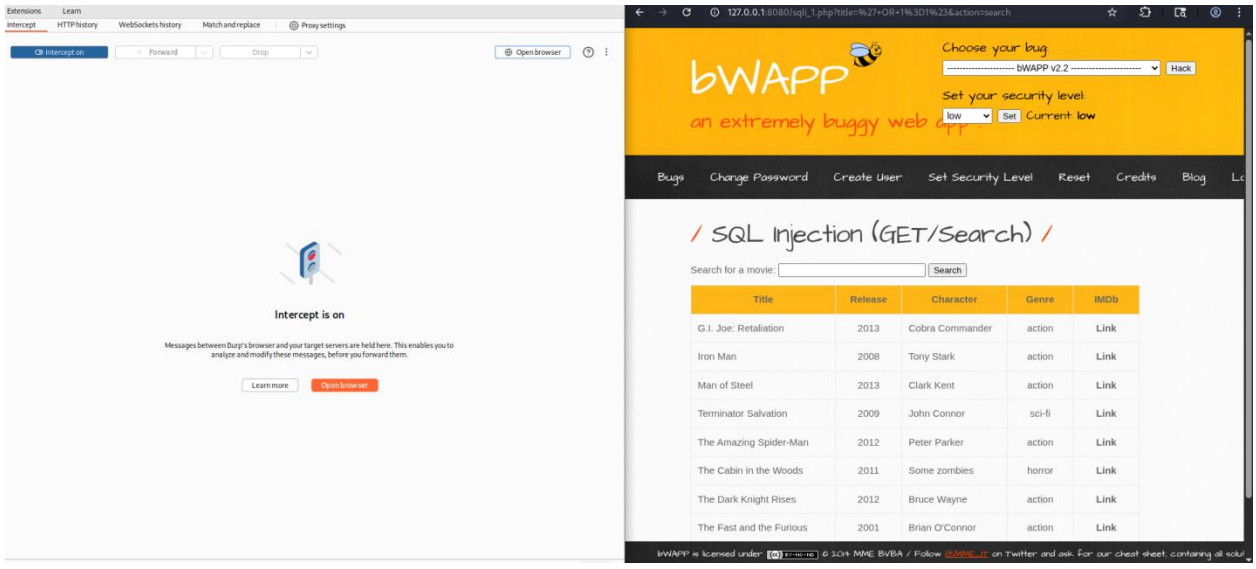
- Screenshot of normal search result;



- Screenshot of Burp intercepted request



- Screenshot showing all movies returned



Impact

An attacker can:

- retrieve all database records
- bypass search filters
- potentially extract sensitive data

Remediation

- Use parameterized queries (prepared statements)
- Validate and sanitize user input
- Avoid directly embedding input in SQL queries

- SQL Injection (POST/Search)

Severity: Critical

Normal Functionality

The application allows users to search for movies using a search form.
When a valid movie name is entered, only matching results are returned.

Vulnerability Description

The application does not validate user input in the POST request body. This allows attackers to inject SQL queries and manipulate database results.

Payload Used: ' OR 1=1#

Steps to Reproduce

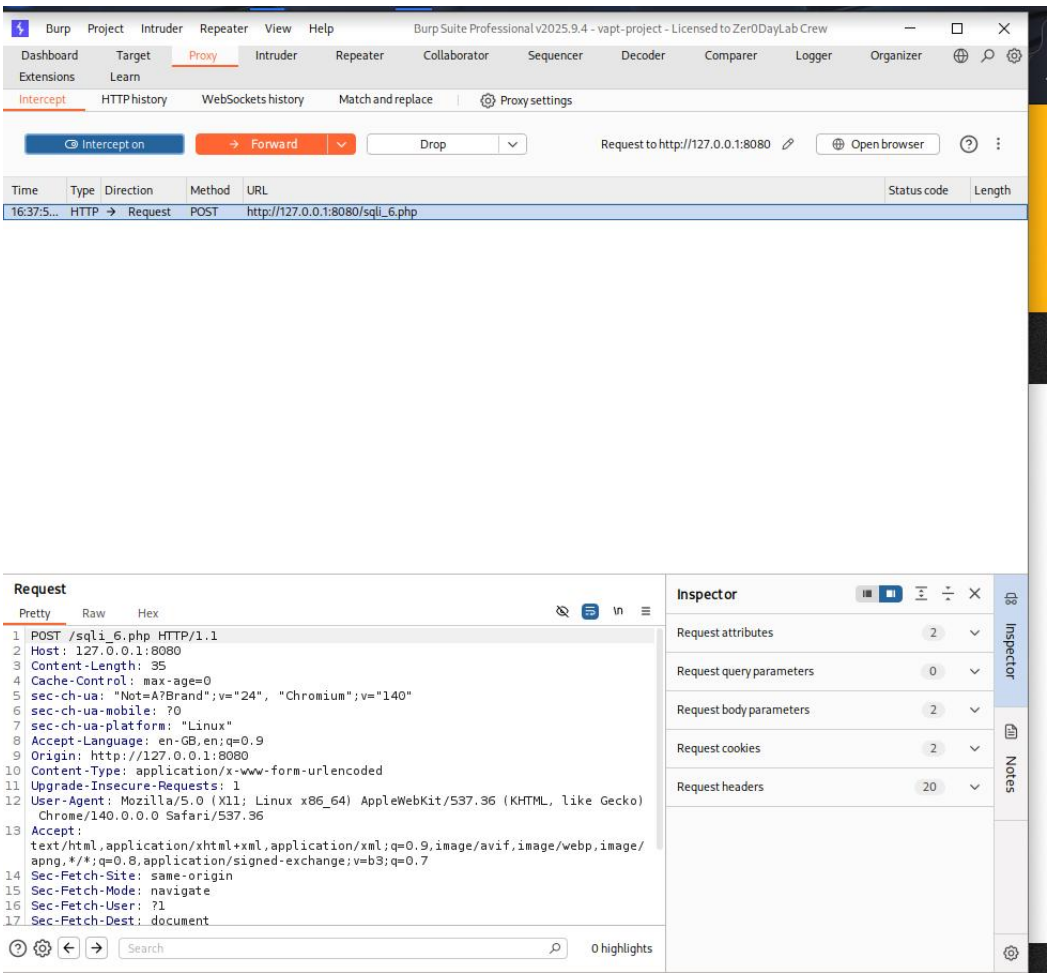
1. Navigate to SQL Injection (POST/Search)
2. Enter the payload ' OR 1=1# in the search field
3. Click Search.
4. Intercept the request using Burp Suite
5. Forward the request

Result

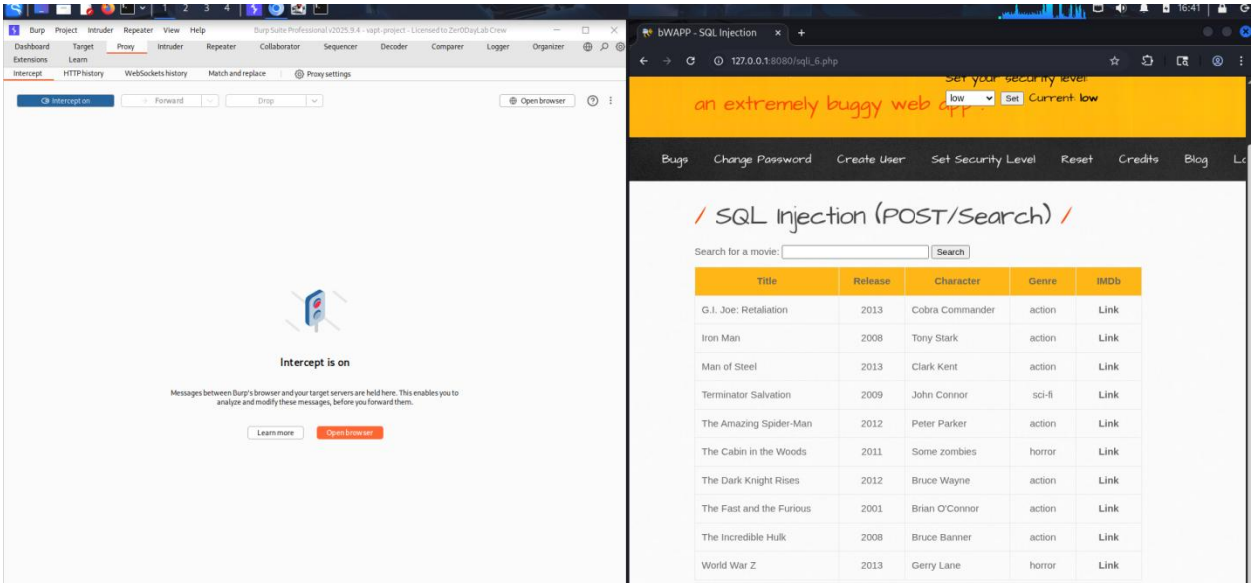
The application returned all movies in the database, indicating that the SQL query logic was bypassed.

Evidence

- Screenshot of Burp POST request;



- Screenshot showing all movies



Impact

An attacker can:

- retrieve all records
- bypass application logic
- access sensitive database content

Remediation

- Use prepared statements
- Validate user input
- Avoid dynamic SQL queries

- SQL Login Form(Hero)

Severity: Critical

Normal Functionality

The application requires valid login credentials (username and password) for authentication. When invalid credentials are entered, access is denied.

Vulnerability Description

The login form does not properly validate user input.
It is vulnerable to SQL Injection, allowing attackers to bypass authentication without valid credentials.

Payload Used: ' OR 1=1#

Steps to Reproduce

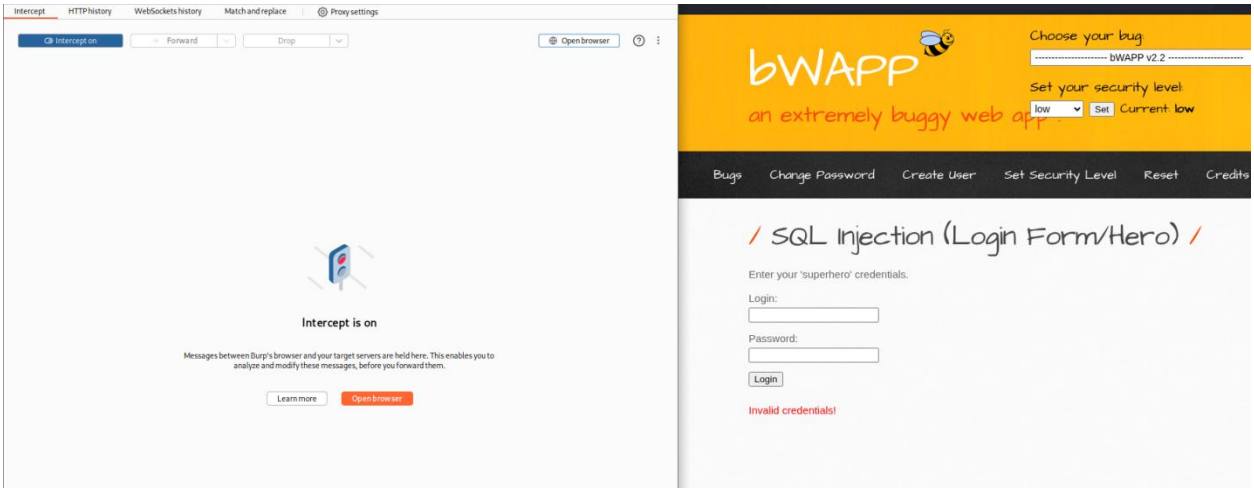
1. Navigate to SQL Injection (Login Form – Hero)
2. Enter ' OR 1=1# in the username field
3. Enter any value in the password field
4. Click **Login**

Result

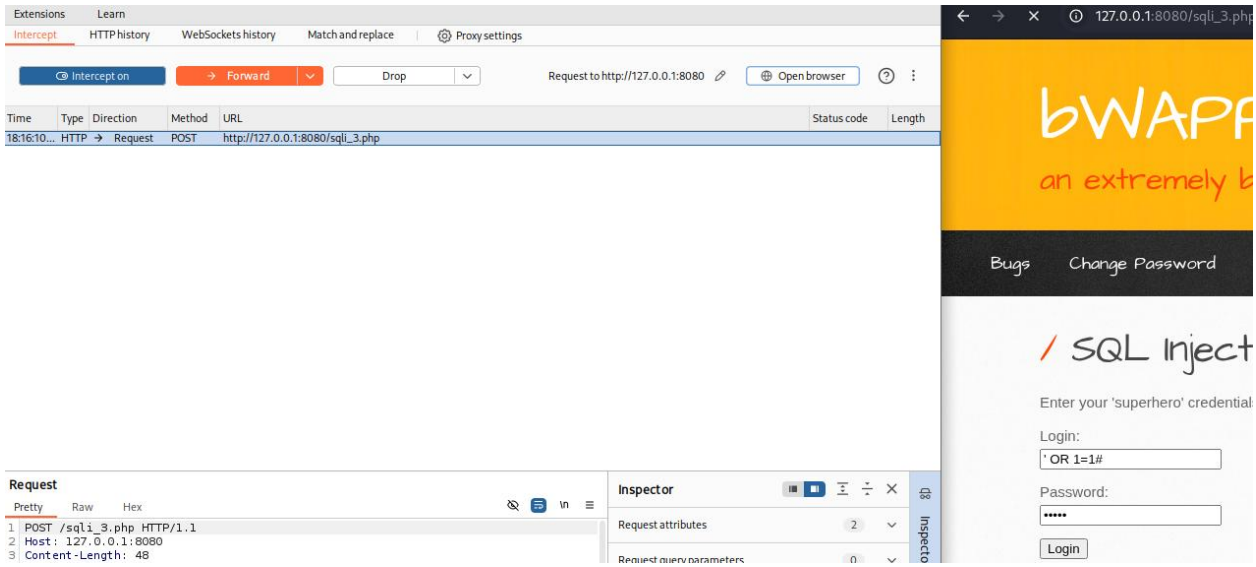
The application grants access and logs the user in as “Neo” without requiring valid credentials.

Evidence

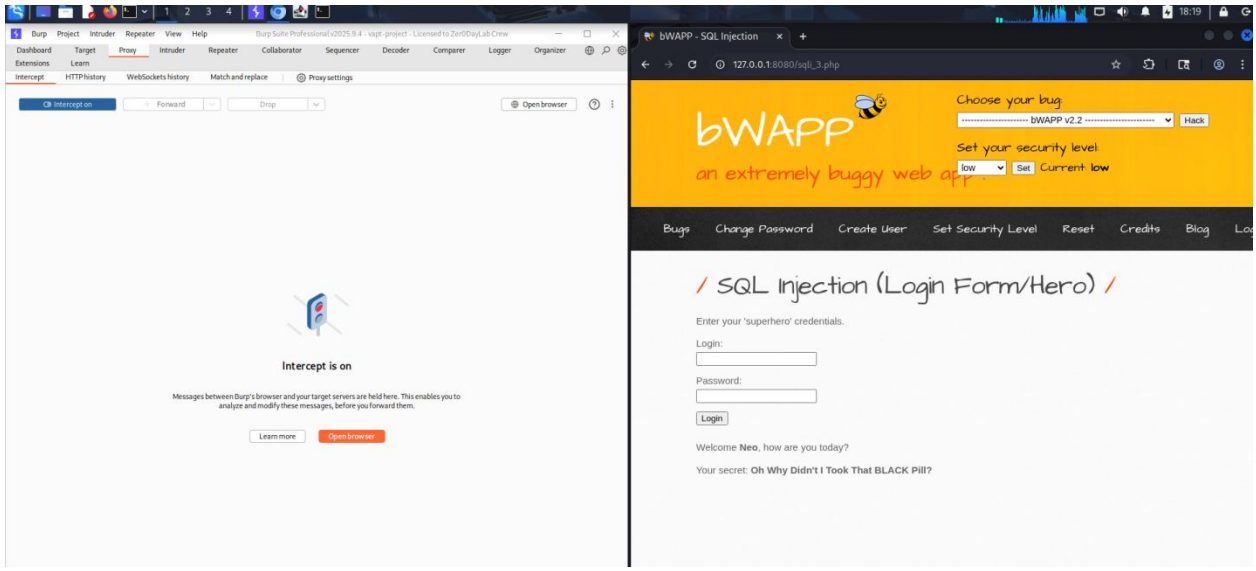
- Screenshot of failed login attempt



- Screenshot of payload input



- Screenshot showing successful login as Neo



Impact

An attacker can:

- bypass authentication
- gain unauthorized access
- impersonate users
- access sensitive data

Remediation

- Use parameterized queries
- Validate and sanitize inputs
- Implement proper authentication checks

- SQL Injection (Stored – Blog)

Severity: High / Critical

Normal Functionality

The application allows users to add entries to a blog. Submitted entries are stored and displayed in a table.

Vulnerability Description

The application does not properly validate user input before storing it in the database.
Malicious input can break the SQL query, leading to SQL Injection.

Payload Used: , ‘Grace');#

Steps to Reproduce

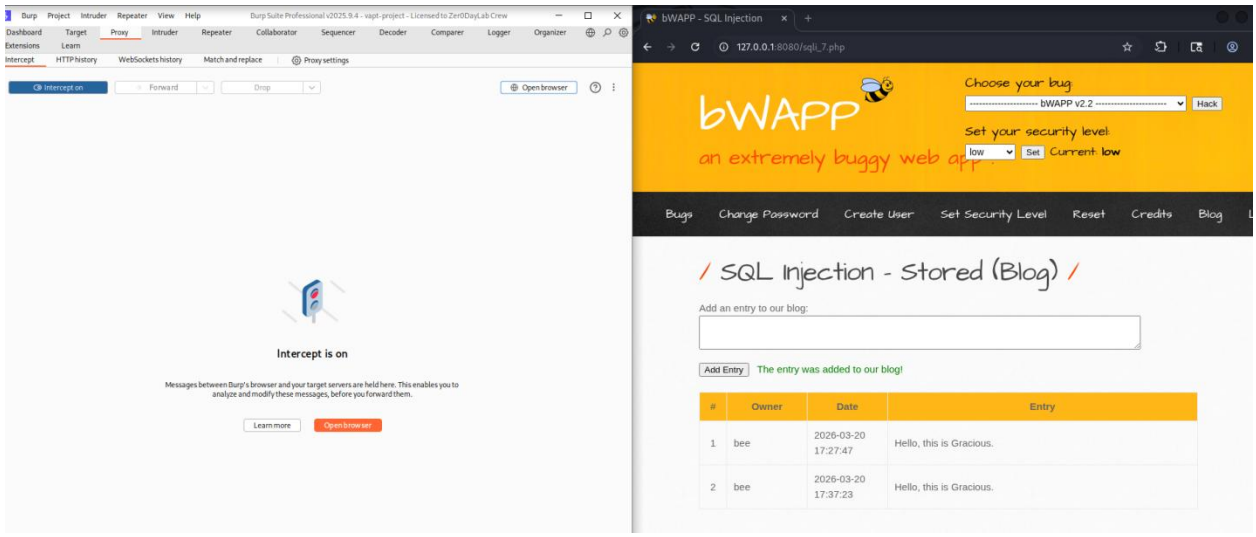
1. Navigate to SQL Injection (Stored – Blog)
2. Enter , ‘Grace');# in the blog entry field
3. Click **Add Entry**

Result

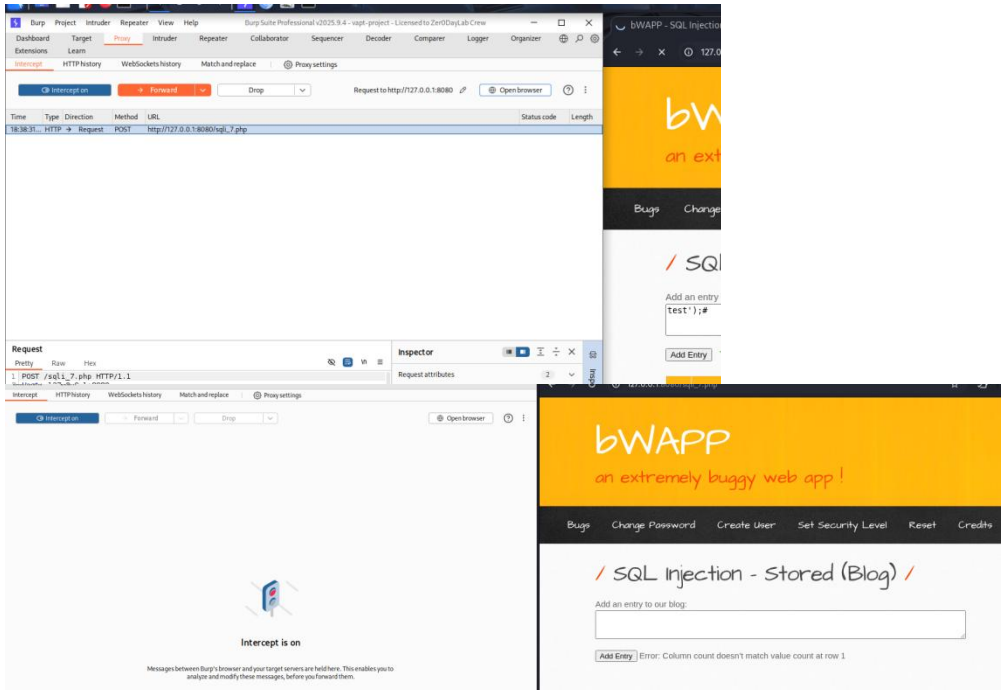
The application returned a database error:
"Column count doesn't match value count at row 1"
This indicates improper handling of user input and confirms SQL Injection.

Evidence

- Screenshot of normal entry

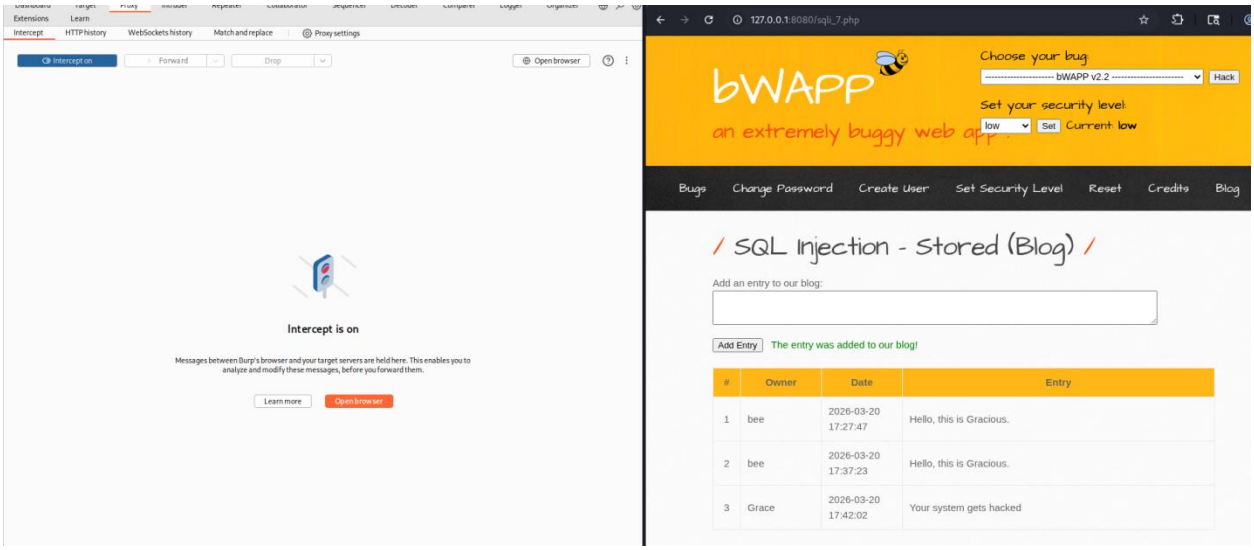


- Screenshot showing SQL error



- Screenshot changing the default username: “bee”

On this page, the user is required to write a blog post, which will then be displayed on the current page. I understand that this page currently only accepts comments and inserts values into the table. Here the username is fixed as 'bee'. So, I have to bypass the logic to store the data that is entered by another user. I entered the above SQL query in the comment field and click Add Entry.



Successfully bypassed the logic.

Impact

An attacker can:

- manipulate database queries
- corrupt stored data
- potentially extract or modify sensitive information

Remediation

- Use parameterized queries
- Validate input before storing
- Implement proper error handling

8.4 OS Command Injection (DNS Lookup)

Severity: Critical

Normal Functionality

The application provides a DNS lookup feature where users can enter a domain name to retrieve DNS information. When a valid domain such as www.nsa.gov is entered, the application performs the lookup and reloads the page without displaying additional system information.

Vulnerability Description

The application does not properly validate user input before passing it to the operating system command. As a result, attackers can inject additional commands that are executed on the server. This vulnerability is known as **OS Command Injection**.

Payload Used: www.nsa.gov; ls -la

Steps to Reproduce

1. Navigate to **Command Injection — DNS Lookup**
2. Enter a normal domain: www.nsa.gov
3. Observe normal behavior
4. Enter the malicious payload: www.nsa.gov; ls -la
5. Click **Lookup**

Result

The application executed the injected command and displayed a list of files along with their permissions.
This confirms that arbitrary system commands can be executed on the server.

Example output observed:

drwxr-xr-x 2 root root 4096 images
-rw-r--r-- 1 root root 1234 index.php

This output shows:

- file names
- permissions
- ownership information

Evidence (Screenshots to include)

1. Normal lookup (www.nsa.gov)

it only refreshes the browser

2. Payload input (www.nsa.gov; ls) and Output showing file list

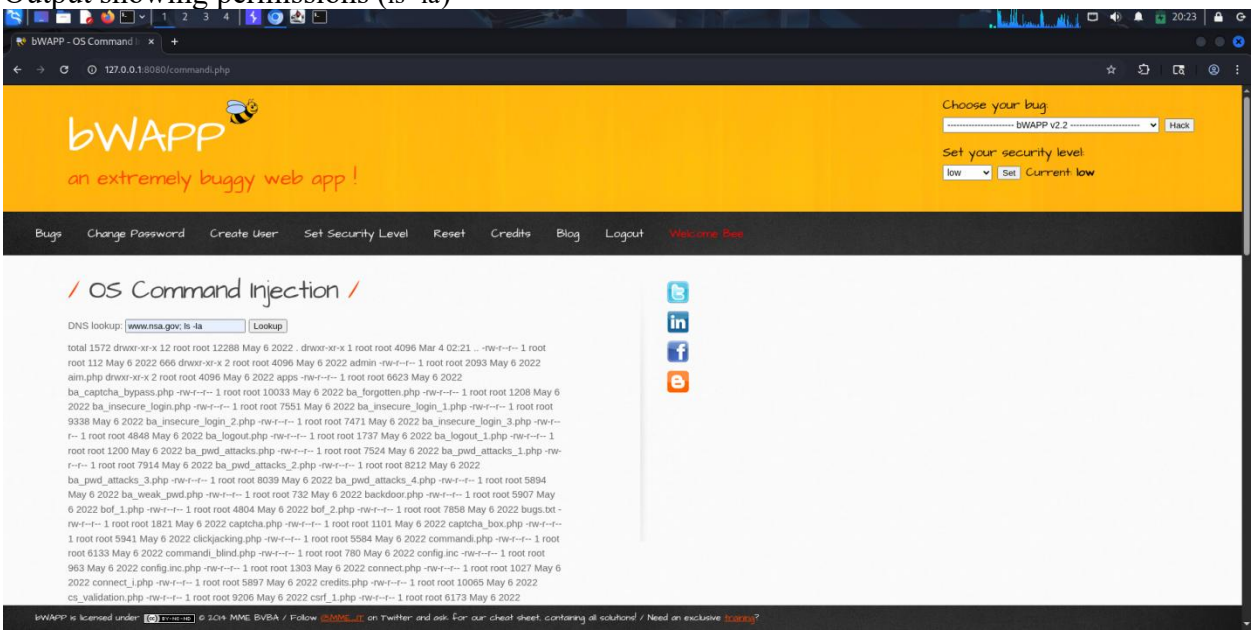
bWAPP - OS Command I

127.0.0.1:8080/commandi.php

DNS lookup: www.nsa.gov; ls Lookup

666 admin aim.php apps ba_captcha_bypass.php ba_forgotten.php ba_insecure_login.php ba_insecure_login_1.php ba_insecure_login_2.php ba_insecure_login_3.php ba_logout.php ba_logout_1.php ba_pwd_attacks.php ba_pwd_attacks_1.php ba_pwd_attacks_2.php ba_pwd_attacks_3.php ba_pwd_attacks_4.php ba_weak_pwd.php backdoor.php bof_1.php bof_2.php bugs.txt captcha.php captcha_box.php clickjacking.php commandi.php commandi_blind.php config.inc config.inc.php connect.php connect_i.php credits.php cs_validation.php csrf_1.php csrf_2.php csrf_3.php db directory_traversal_1.php directory_traversal_2.php documents fonts functions_external.php heartbleed.php hostheader_1.php hostheader_2.php hpp-1.php hpp-2.php hpp-3.php htmli_current_url.php htmli_get.php htmli_post.php htmli_stored.php http_response_splitting.php http_verb_tampering.php iframei.php images index.php info.php info_install.php information_disclosure_1.php information_disclosure_2.php information_disclosure_3.php information_disclosure_4.php insecure_crypt_storage_1.php insecure_crypt_storage_2.php insecure_crypt_storage_3.php insecure_direct_object_ref_1.php insecure_direct_object_ref_2.php insecure_direct_object_ref_3.php insecure_iframe.php install.php insuff_transp_layer_protect_1.php insuff_transp_layer_protect_2.php insuff_transp_layer_protect_3.php insuff_transp_layer_protect_4.php js lang_en.php lang_fr.php lang_nl.php ldap_connect.php ldapi.php lfi_sqlitemanager.php login.php logout.php maili.php manual_interv.php message.txt password_change.php passwords php_cgi.php php_eval.php phpi.php phpi_sqlitemanager.php phpinfo.php portal.bak portal.php portal.zip reset.php restrict_device_access.php restrict_folder_access.php rlfi.php robots.txt secret-cors-1.php secret-cors-2.php secret-cors-3.php secret.php secret_change.php secret_html.php security.php security_level_check.php security_level_set.php selections.php shellshock.php shellshock.sh sm_cors.php sm_cross_domain_policy.php sm_dos_1.php sm_dos_2.php sm_dos_3.php sm_dos_4.php sm_ftp.php sm_local_priv_esc_1.php sm_local_priv_esc_2.php sm_mitm_1.php sm_mitm_2.php sm_obu_files.php sm_robots.php sm_samba.php sm_snmp.php sm_webdav.php sm_xst.php smgmt_admin_portal.php smgmt_cookies_httponly.php smgmt_cookies_secure.php smgmt_sessionid_url.php smgmt_strong_sessions.php soap sqli_1.php sqli_10-1.php sqli_10-2.php sqli_11.php sqli_12.php sqli_13-ps.php sqli_13.php sqli_14.php sqli_15.php sqli_16.php sqli_17.php sqli_2-ps.php sqli_2.php sqli_3.php sqli_4.php sqli_5.php sqli_6.php sqli_7.php sqli_8-1.php sqli_8-2.php sqli_9.php sqli_drupal.php ssii.php ssrf.php stylesheets test.php top_security.php training.php training_install.php unrestricted_file_upload.php unvalidated_redir_fwd_1.php unvalidated_redir_fwd_2.php update.php user_activation.php user_extra.php user_new.php web.config ws_soap.php xmli_1.php xmli_2.php xss_ajax_1-1.php xss_ajax_1-2.php xss_ajax_2-1.php xss_ajax_2-2.php xss_back_button.php xss_custom_header.php xss_eval.php xss_get.php xss_href-1.php xss_href-2.php xss_href-3.php xss_json.php xss_login.php xss_php_self.php xss_phpmyadmin.php xss_post.php xss_referer.php xss_sqlitemanager.php xss_stored_1.php xss_stored_2.php xss_stored_3.php xss_stored_4.php xss_user_agent.php xxe-1.php xxe-2.php

3. Output showing permissions (ls -la)



4. command like whoami

Impact

An attacker can execute arbitrary commands on the server.
This could allow the attacker to:

- access sensitive files
- modify system data
- escalate privileges
- take full control of the server

Remediation

The application should validate and sanitize user input before executing system commands. Recommended controls include:

- Use input validation and allow-lists
- Avoid passing user input directly to system commands
- Use secure APIs instead of shell execution
- Implement least privilege for system processes.

8.5 Cross-Site Request Forgery (CSRF)

- **CSRF — Change Password**

Severity: Medium

Normal Functionality

The application allows a logged-in user to change their password by submitting a form with a new password and confirmation.

Vulnerability Description

The password change functionality does not include a CSRF protection mechanism such as a CSRF token. This means an attacker can trick a logged-in user into submitting a malicious request without their knowledge.

Request Captured

POST /csrf_1.php
password_new=123
password_conf=123

Steps to Reproduce

1. Log in to the application.
2. Navigate to the Change Password page.
3. Intercept the request using Burp Suite.
4. Submit a new password.
5. Observe that the request does not contain a CSRF token.

Result

The password was successfully changed without any CSRF token validation.

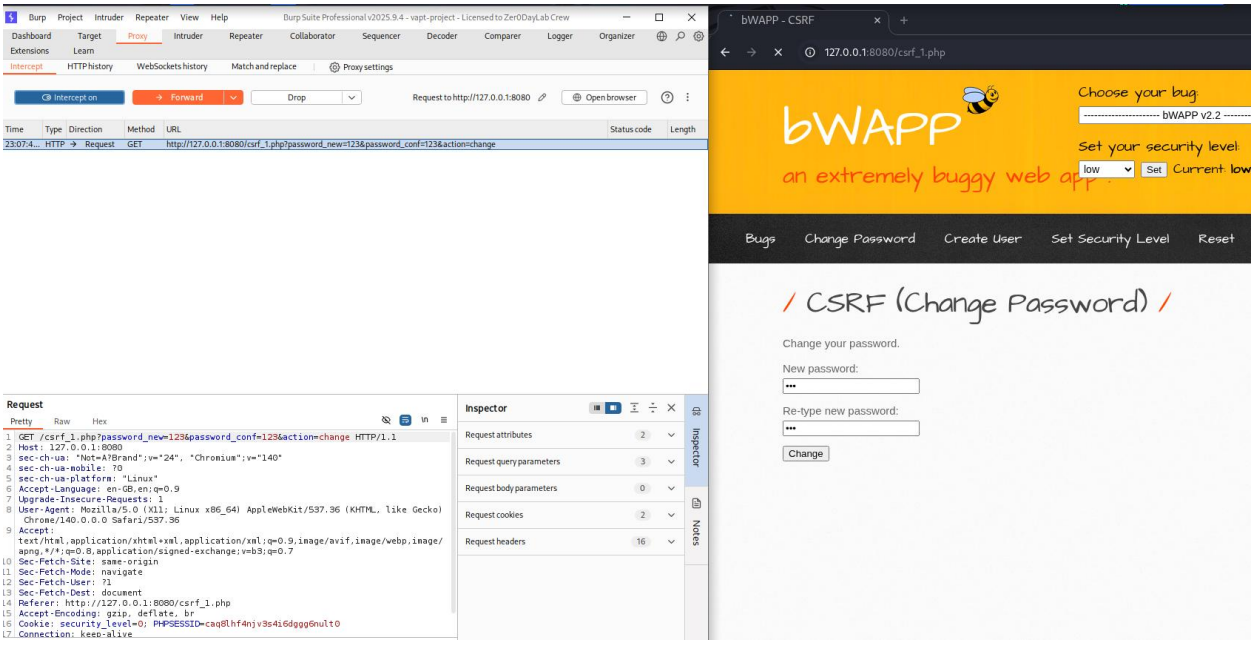
Impact

An attacker can:

- Change a user's password
- Perform unauthorized actions
- Hijack user accounts

Remediation

- Implement CSRF tokens for sensitive actions
- Validate tokens on the server
- Use Same Site cookies



- Captured password change request without CSRF token

- Sensitive action (password change)
- Request can be executed without verification token
- Therefore vulnerable to CSRF

- **CSRF— Change Secret**

Severity: Medium

Normal Functionality

The application allows a logged-in user to change their secret value by submitting a form.
When a new secret is entered and submitted, the application updates the user's secret successfully.

Vulnerability Description

The change secret functionality does not implement proper CSRF protection.
The request can be executed without a CSRF token or additional verification, allowing attackers to perform unauthorized actions on behalf of a logged-in user.

Payload / Request Captured

POST /csrf_2.php HTTP/1.1

secret=new

Steps to Reproduce

1. Log in to the application.
2. Navigate to **CSRF — Change Secret** page.
3. Enter a new secret value.
4. Intercept the request using Burp Suite.
5. Observe that the request does not contain a CSRF token.

Result

The secret value was successfully changed without any CSRF protection mechanism.
This confirms that the application is vulnerable to Cross-Site Request Forgery.

Evidence (Screenshots)

Include:

1. Change Secret page
2. Burp intercepted POST request
3. Parameters showing secret=new
4. Absence of CSRF token

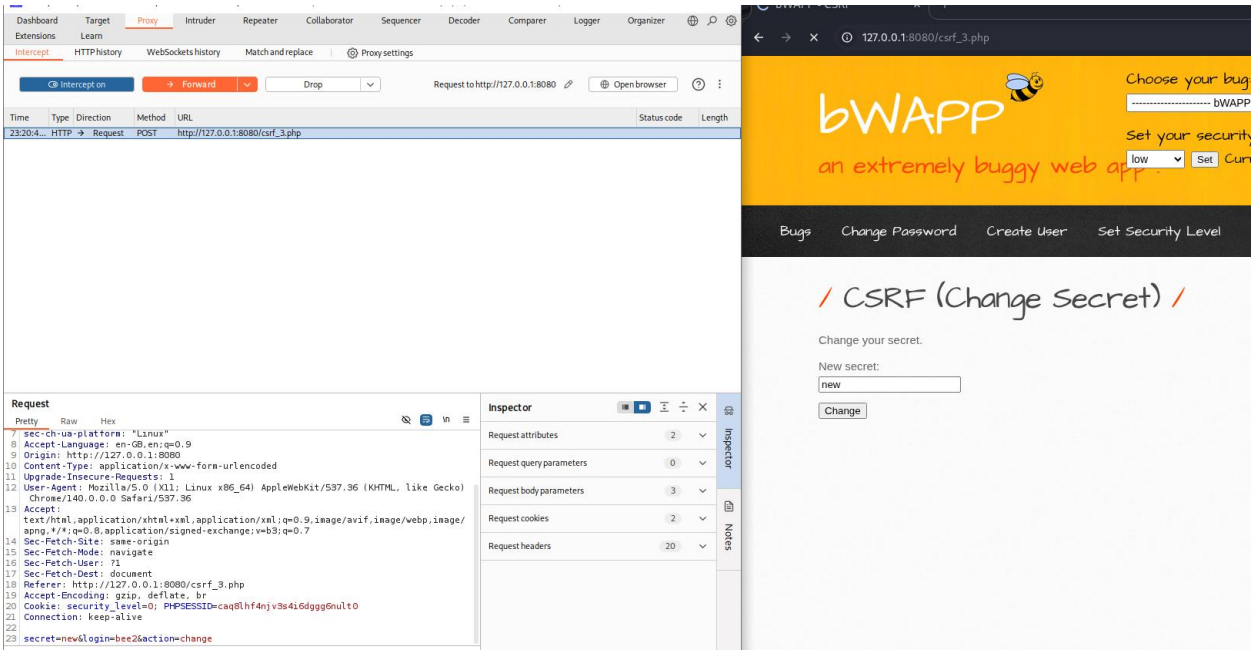
Impact

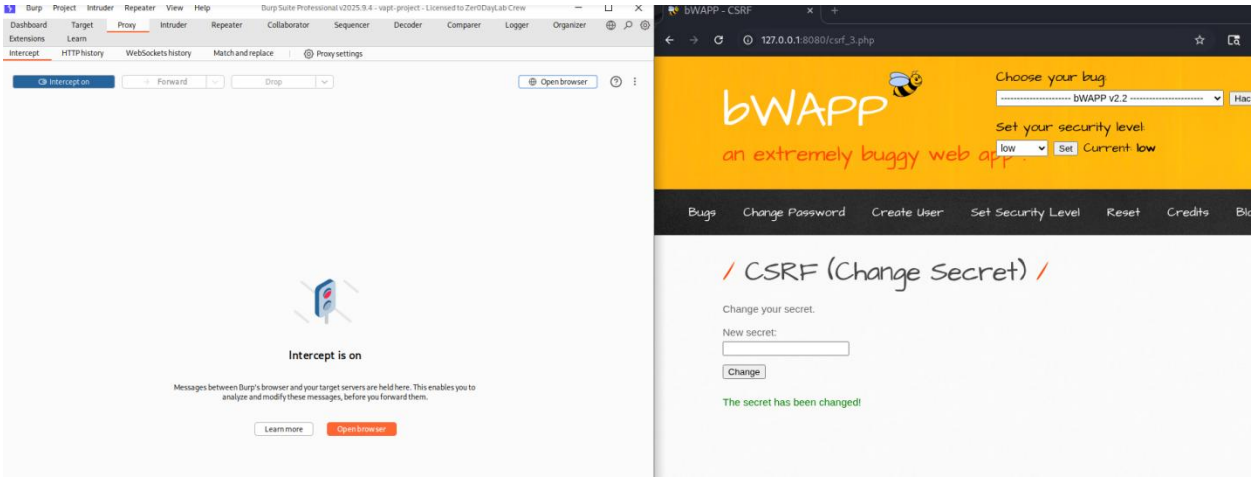
An attacker can:

- Modify sensitive user information
- Perform actions without user consent
- Exploit authenticated sessions

Remediation

- Implement CSRF tokens for all sensitive operations
- Validate tokens on the server
- Use SameSite cookie attributes
- Require user confirmation for critical changes





8.6 Broken Authentication Insecure Login Form

Severity: High

Normal Functionality

The application provides a login form that requires users to enter a username and password to access restricted areas.
Normally, authentication credentials should be securely stored and not visible to users.

Vulnerability Description

The login credentials are exposed in the client-side source code of the login page.
An attacker can view the page source and retrieve valid login credentials without authorization.

This represents a **Broken Authentication** vulnerability because sensitive authentication information is improperly disclosed.

Discovery Method

The page source code was inspected using:

Right-click → View Page Source

Credentials Found

Username: tonyshark
Password: I am Iron Man

Steps to Reproduce

1. Navigate to **Broken Authentication — Insecure Login Form**
2. Right-click on the login page
3. Select **View Page Source**
4. Search for login credentials in the HTML code
5. Use the discovered credentials to log in

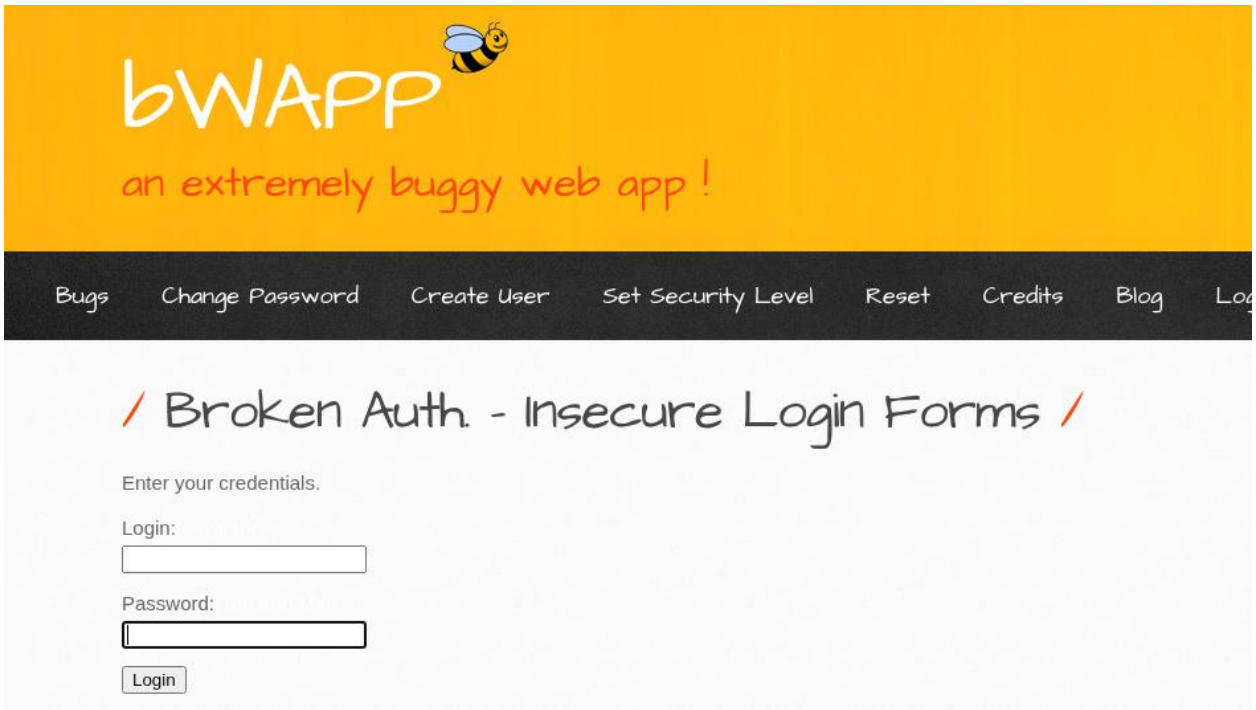
Result

Valid login credentials were visible in the page source, allowing unauthorized users to authenticate successfully.

Evidence (Screenshots)

Include:

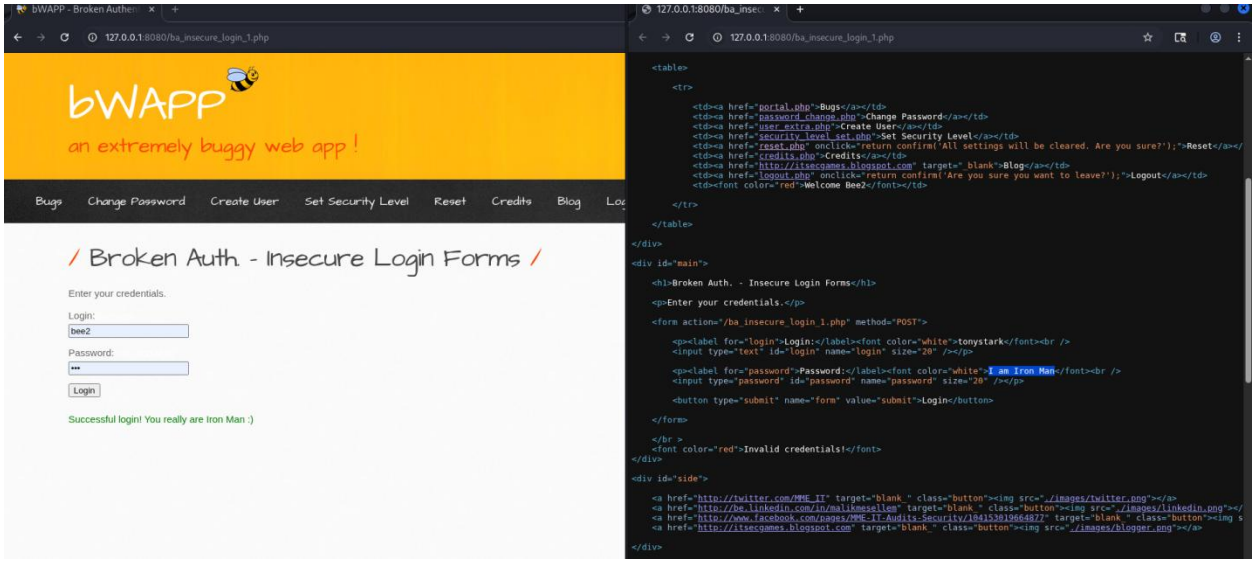
1. Login page



2. Page source showing credentials

```
<h1>Broken Auth. - Insecure Login Forms</h1>

<p>Enter your credentials.</p>

<form action="/ba_insecure_login_1.php" method="POST">

    <p><label for="login">Login:</label><font color="white">tonystark</font><br />
    <input type="text" id="login" name="login" size="20" /></p>

    <p><label for="password">Password:</label><font color="white">I am Iron Man</font><br />
    <input type="password" id="password" name="password" size="20" /></p>

    <button type="submit" name="form" value="submit">Login</button>

</form>
```

3. Successful login using exposed credentials



Impact

An attacker can:

- Obtain valid login credentials
- Gain unauthorized access
- Compromise user accounts
- Access sensitive information

Remediation

- Do not store credentials in client-side code
- Use secure authentication mechanisms
- Implement proper credential management
- Store sensitive data on the server side only

9. Risk Severity Summary

The following table summarizes the vulnerabilities identified during the Vulnerability Assessment and Penetration Testing (VAPT) of the bWAPP web application. Each vulnerability is classified based on its severity level and potential impact on the system.

S/N	Vulnerability	Category	Severity	Impact
1	HTML Injection — Reflected (GET)	Injection	Low	Allows attackers to modify webpage content

2	Cross-Site Scripting (XSS) — Reflected (GET)	Injection	Medium	Allows execution of malicious scripts in user bro
3	Cross-Site Scripting (XSS) — Stored (User-Agent)	Injection	High	Affects multiple users and persists on server
4	SQL Injection — GET / Search	Injection	Critical	Allows retrieval of all database records
5	SQL Injection — POST / Search	Injection	Critical	Allows manipulation of database queries
6	SQL Injection — Login Form (Hero)	Authentication Bypass	Critical	Allows unauthorized login access
7	SQL Injection — Stored (Blog)	Injection	High	Allows database manipulation and data corruptio
8	OS Command Injection — DNS Lookup	Command Injection	Critical	Allows execution of system commands
9	CSRF — Change Password	CSRF	Medium	Allows unauthorized password change
10	CSRF — Change Secret	CSRF	Medium	Allows unauthorized modification of user data
11	Broken Authentication — Insecure Login Form	Authentication	High	Exposes valid login credentials

10. Remediation Recommendations

The vulnerabilities identified during the Vulnerability Assessment and Penetration Testing (VAPT) of the bWAPP web application highlight weaknesses in input validation, authentication mechanisms, and security controls. Implementing the following remediation measures will significantly improve the security posture of the application and reduce the risk of exploitation.

1. **Input Validation and Output Encoding**

All user inputs should be properly validated and sanitized before processing or displaying them in the application. Output encoding should be applied to prevent the execution of malicious scripts in the browser.

Recommended Controls:

- Validate all user input on the server side.
- Sanitize input to remove malicious characters.
- Encode output before rendering in the browser.
- Implement Content Security Policy (CSP).

This will help prevent vulnerabilities such as:

- HTML Injection
- Cross-Site Scripting (XSS)
- SQL Injection

2. Secure Database Interaction

Database queries should be implemented using secure coding practices to prevent unauthorized manipulation of database operations.

Recommended Controls:

- Use parameterized queries (prepared statements)
- Avoid dynamic SQL queries
- Validate and sanitize database inputs
- Implement proper error handling
- Restrict database permissions

This will help prevent:

- SQL Injection
- Data manipulation
- Unauthorized database access

3. Command Execution Security

Applications should avoid executing system commands directly using user input. Secure methods should be used to handle system-level operations.

Recommended Controls:

- Validate and restrict user input.
- Use allow-listed commands only.
- Avoid passing user input directly to system commands.
- Implement least privilege for system processes.

This will help prevent:

- OS Command Injection
- Unauthorized system access.

4. Cross-Site Request Forgery (CSRF) Protection

Sensitive operations such as password changes and account updates should include protection mechanisms to ensure that requests are legitimate.

Recommended Controls:

- Implement CSRF tokens for all sensitive actions.
- Validate tokens on the server.
- Use SameSite cookie attributes.
- Require user confirmation for critical actions.

This will help prevent:

- Unauthorized account actions.
- Session abuse.

5. Authentication and Credential Security

Authentication mechanisms should be strengthened to prevent unauthorized access and exposure of sensitive login credentials.

Recommended Controls:

- Store credentials securely on the server side.
- Never expose credentials in client-side code.
- Enforce strong password policies.
- Implement account lockout after multiple failed login attempts.
- Use secure session management.

This will help prevent:

- Broken Authentication
- Credential exposure.
- Unauthorized access.

11. Conclusion

The Vulnerability Assessment and Penetration Testing (VAPT) conducted on the **bWAPP (Buggy Web Application)** successfully identified multiple security vulnerabilities within the application in a controlled laboratory environment. The assessment demonstrated how weaknesses in input validation, authentication mechanisms, and security controls can be exploited to compromise the confidentiality, integrity, and availability of a web application system.

Throughout the testing process, several vulnerabilities aligned with the **OWASP Top 10** security risks were identified and exploited. These included **HTML Injection**, **Cross-Site Scripting (XSS)**, **SQL Injection**, **OS Command Injection**, **Cross-Site Request Forgery (CSRF)**, and **Broken Authentication**. Each vulnerability was carefully tested using structured penetration testing techniques, and evidence of successful exploitation was documented using screenshots and captured HTTP requests.

The findings from this assessment highlight the importance of implementing secure coding practices, proper input validation, strong authentication controls, and continuous security testing to protect web applications from potential cyber threats. If these vulnerabilities were present in a real production environment, attackers could exploit them to gain unauthorized access, manipulate system behavior, or access sensitive data.

By addressing the identified vulnerabilities and implementing the recommended remediation measures outlined in this report, organizations can significantly improve the security posture of their web applications and reduce the risk of successful cyber attacks.

Overall, this assessment demonstrates the value of proactive security testing and reinforces the need for regular vulnerability assessments and penetration testing as part of a comprehensive cybersecurity strategy.